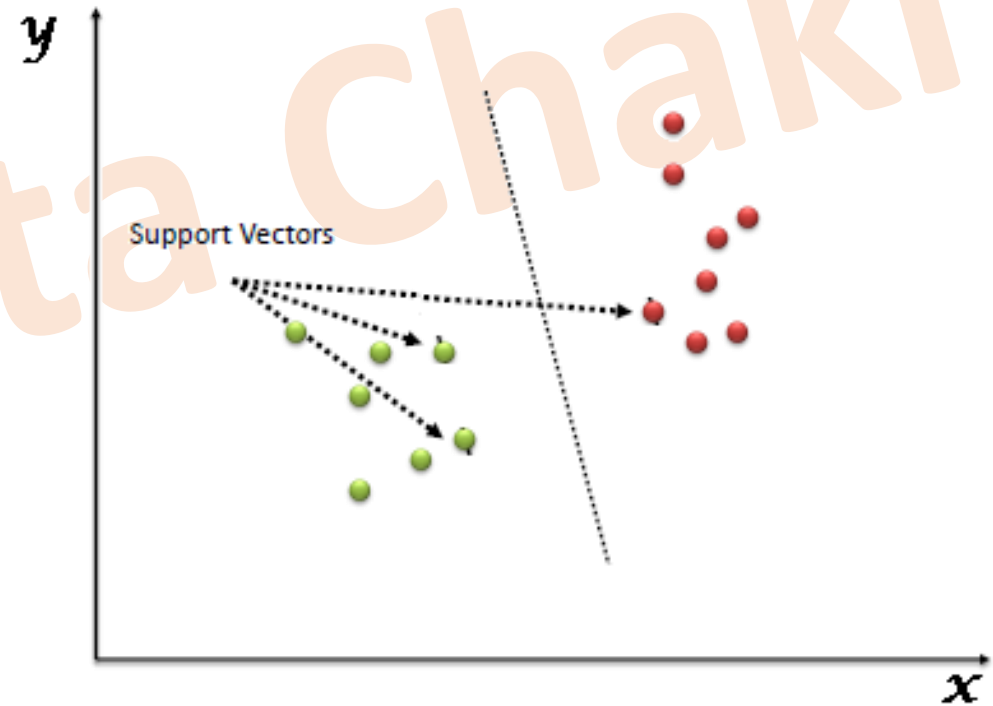


Support Vector Machine

Dr. Jyotismita Chaki

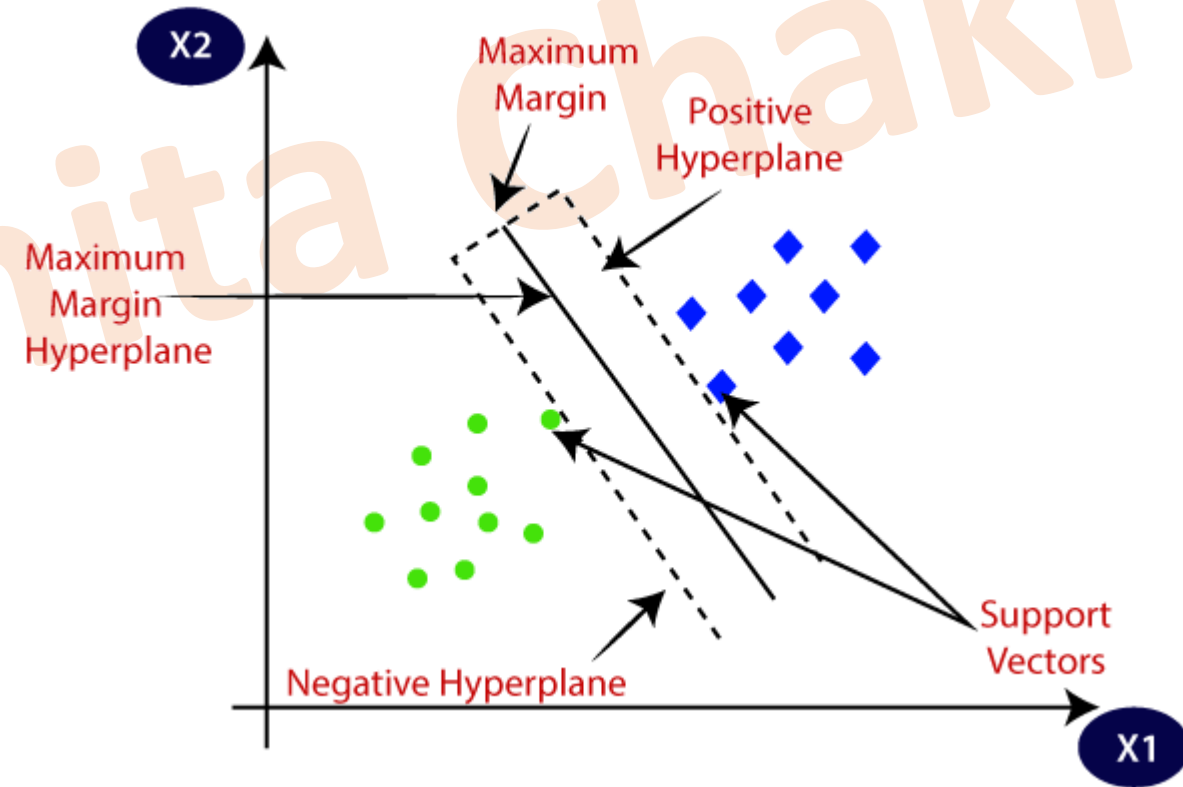
Support Vector Machine

- A *Support Vector Machine* (SVM) is a very powerful and versatile Machine Learning model, capable of performing linear or nonlinear classification, regression, and even outlier detection.
- It is one of the most popular models in Machine Learning, and anyone interested in Machine Learning should have it in their toolbox.
- SVMs are particularly well suited for classification of complex but small- or medium-sized datasets.



Support Vector Machine

- The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.
- SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine.



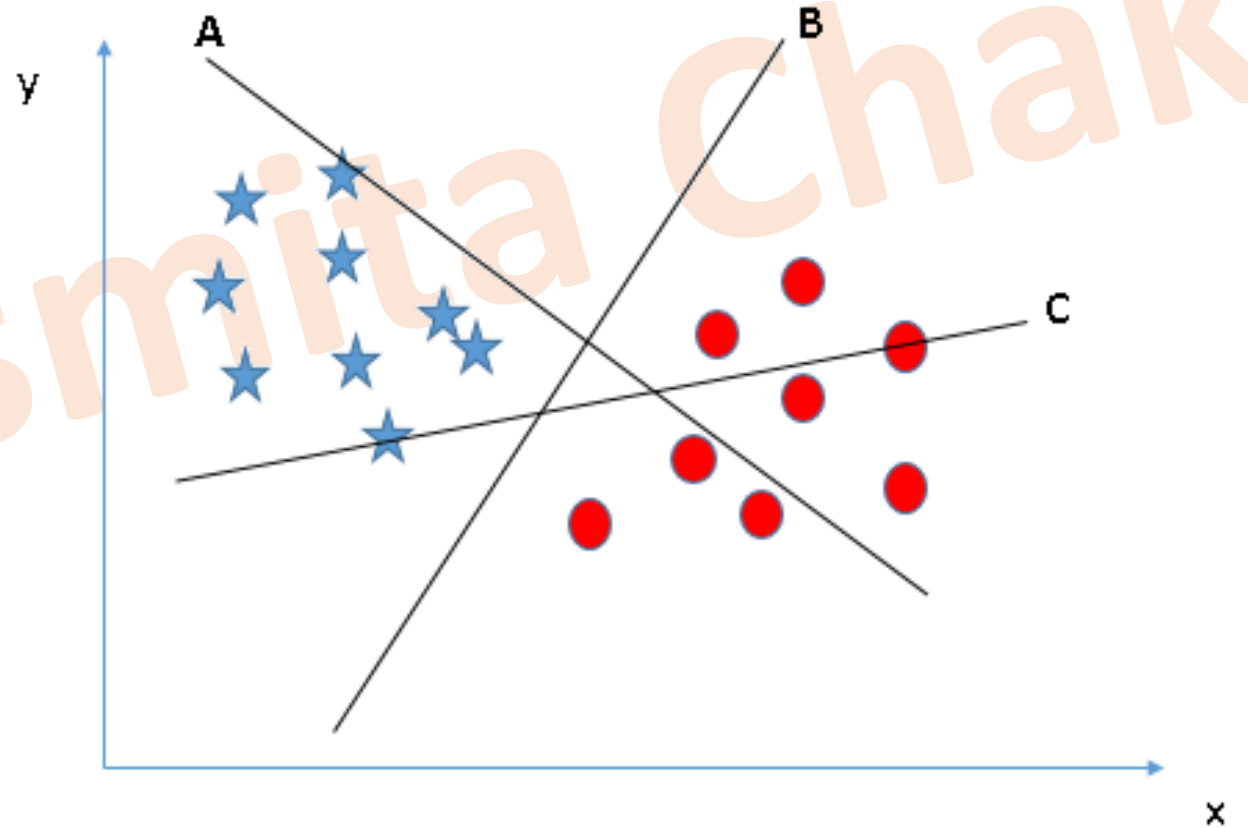
Support Vector Machine: Types

- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

Support Vector Machine: Linear Hyperplane: How does it work?

- **Identify the right hyper-plane (Scenario-1):**

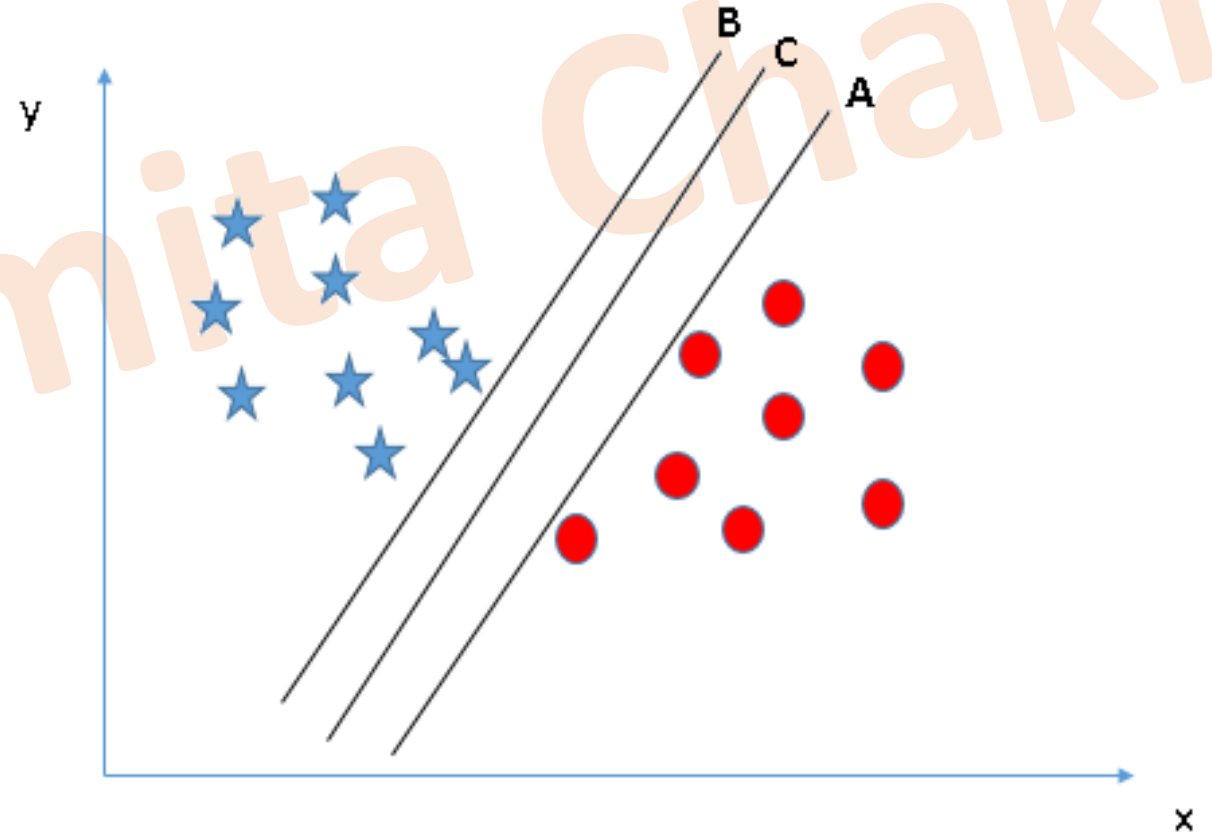
- Here, we have three hyper-planes (A, B, and C). Now, identify the right hyper-plane to classify stars and circles.
- You need to remember a thumb rule to identify the right hyper-plane: “Select the hyper-plane which segregates the two classes better”. In this scenario, hyper-plane “B” has excellently performed this job.



Support Vector Machine: Linear Hyperplane: How does it work?

- **Identify the right hyper-plane (Scenario-2):**

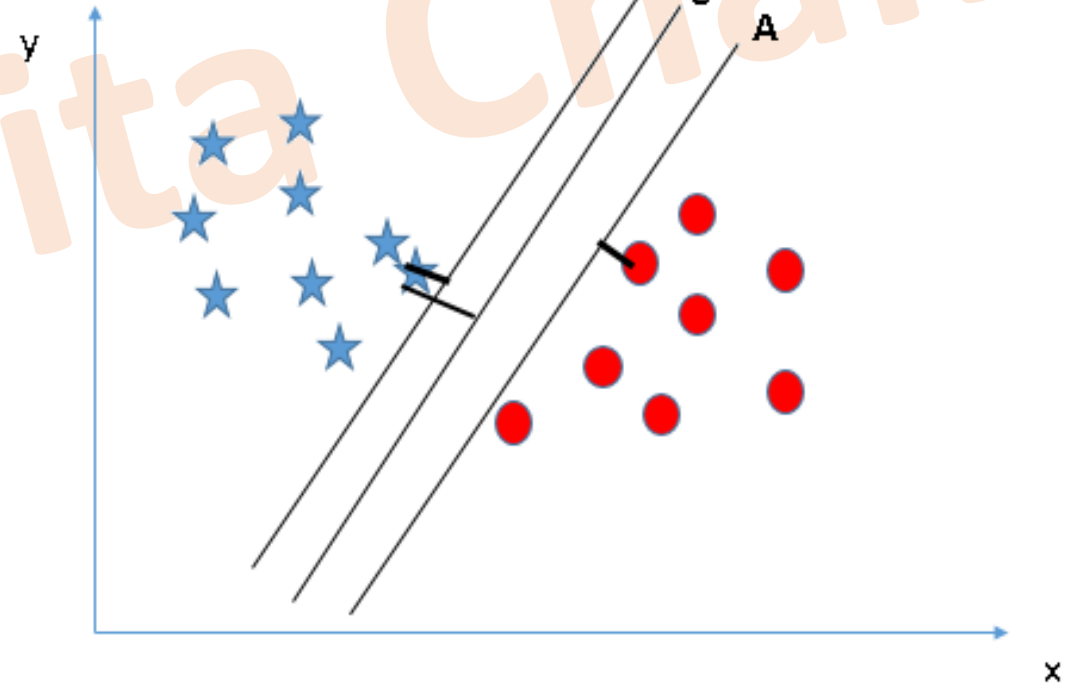
- Here, we have three hyper-planes (A, B, and C) and all are segregating the classes well. Now, How can we identify the right hyper-plane?



Support Vector Machine: Linear Hyperplane: How does it work?

- **Identify the right hyper-plane (Scenario-2):**

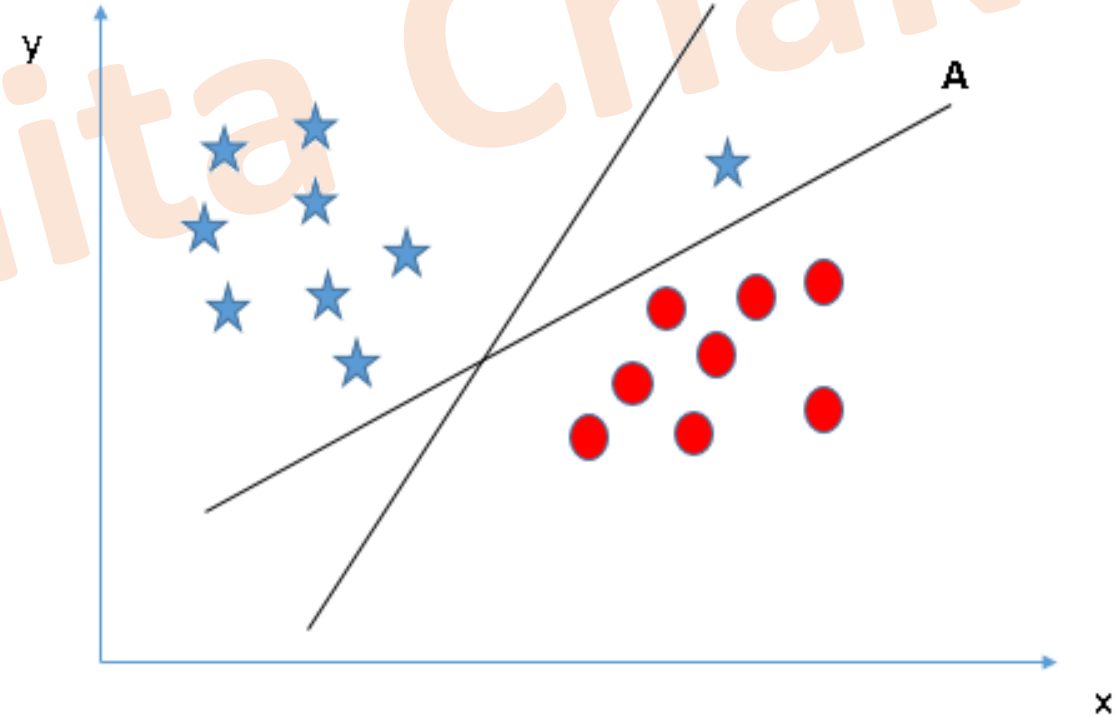
- Here, maximizing the distances between nearest data point (either class) and hyper-plane will help us to decide the right hyper-plane. This distance is called as **Margin**.
- Above, you can see that the margin for hyper-plane C is high as compared to both A and B. Hence, we name the right hyper-plane as C.
- Another lightning reason for selecting the hyper-plane with higher margin is robustness.
- If we select a hyper-plane having low margin then there is high chance of misclassification.



Support Vector Machine: Linear Hyperplane: How does it work?

- **Identify the right hyper-plane (Scenario-3):**

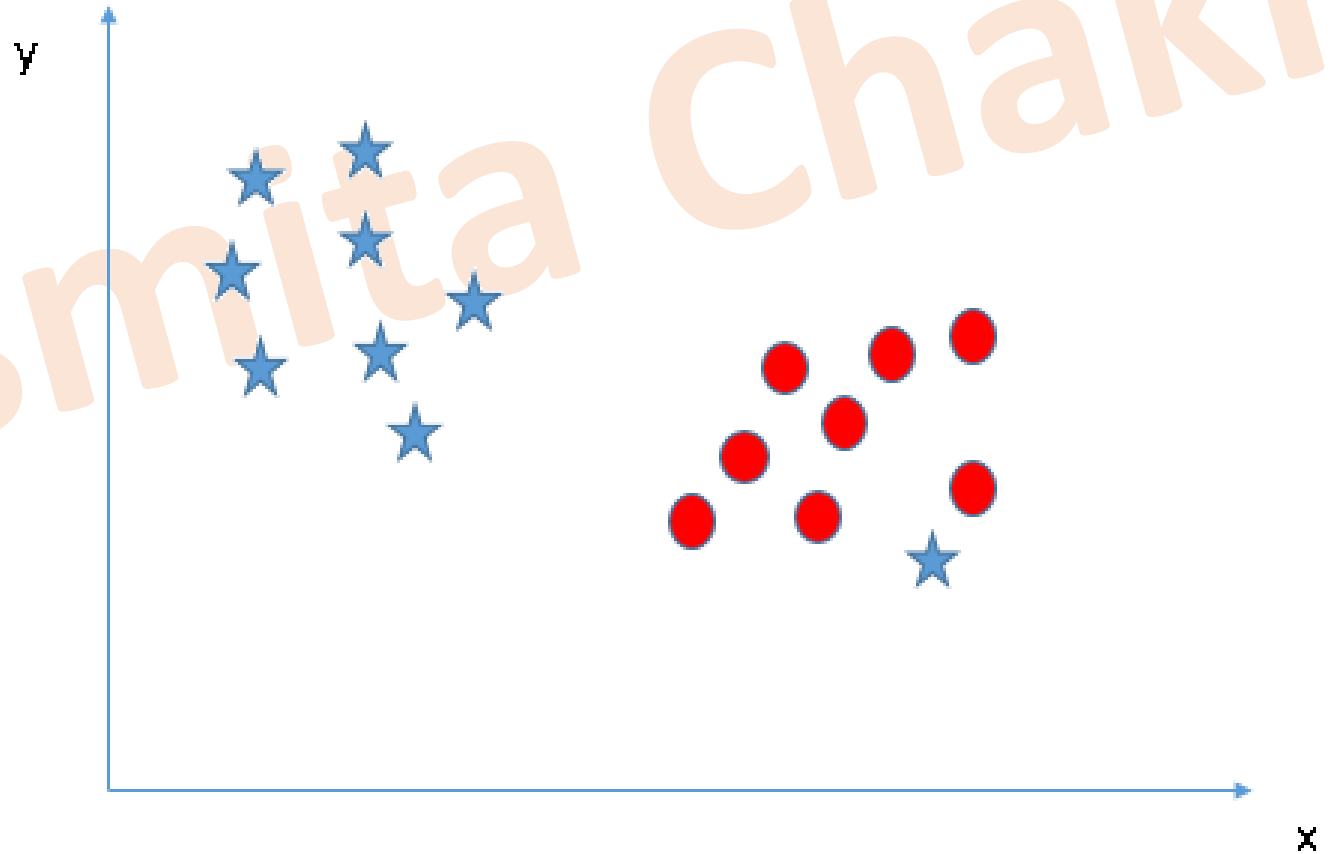
- Hint: Use the rules as discussed in previous section to identify the right hyper-plane
- SVM selects the hyper-plane which classifies the classes accurately prior to maximizing margin. Here, hyper-plane B has a classification error and A has classified all correctly. Therefore, the right hyper-plane is **A**.



Support Vector Machine: Linear Hyperplane: How does it work?

- **Can we classify two classes (Scenario-4)?:**

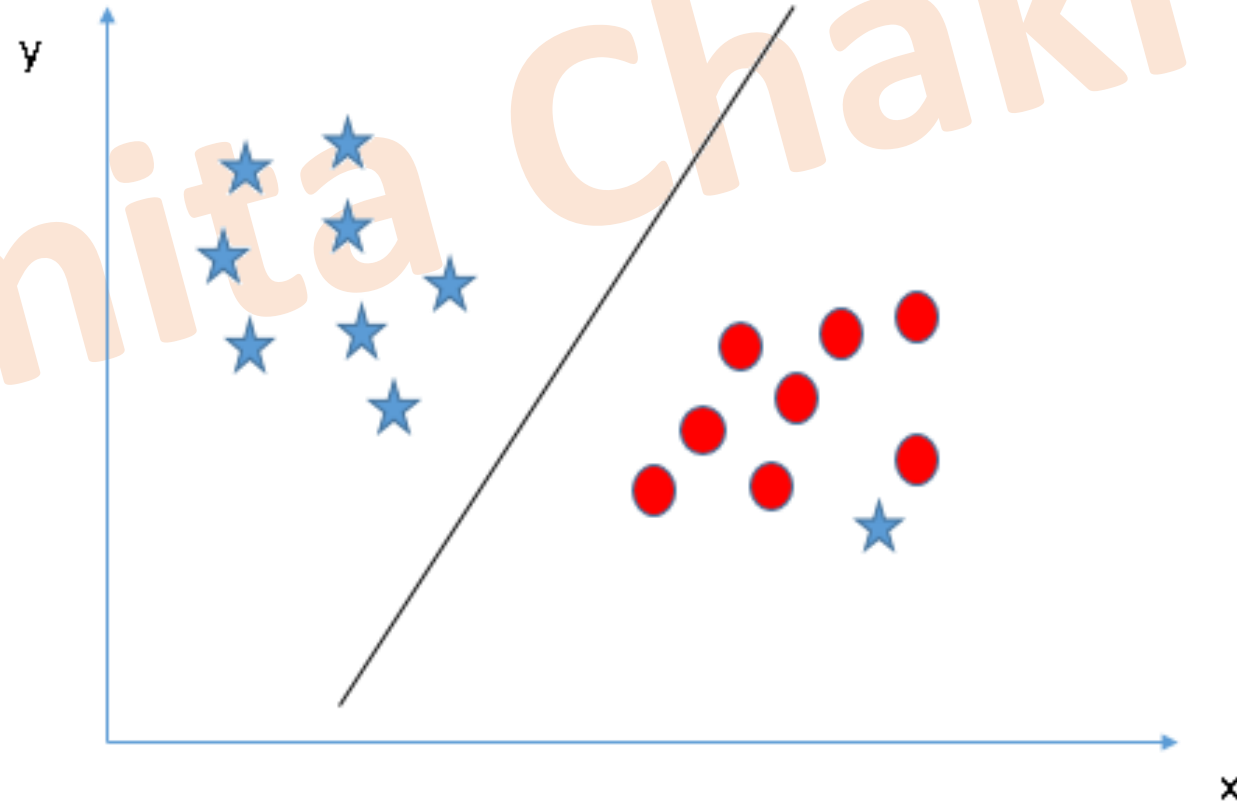
- Below, I am unable to segregate the two classes using a straight line, as one of the stars lies in the territory of other(circle) class as an outlier.



Support Vector Machine: Linear Hyperplane: How does it work?

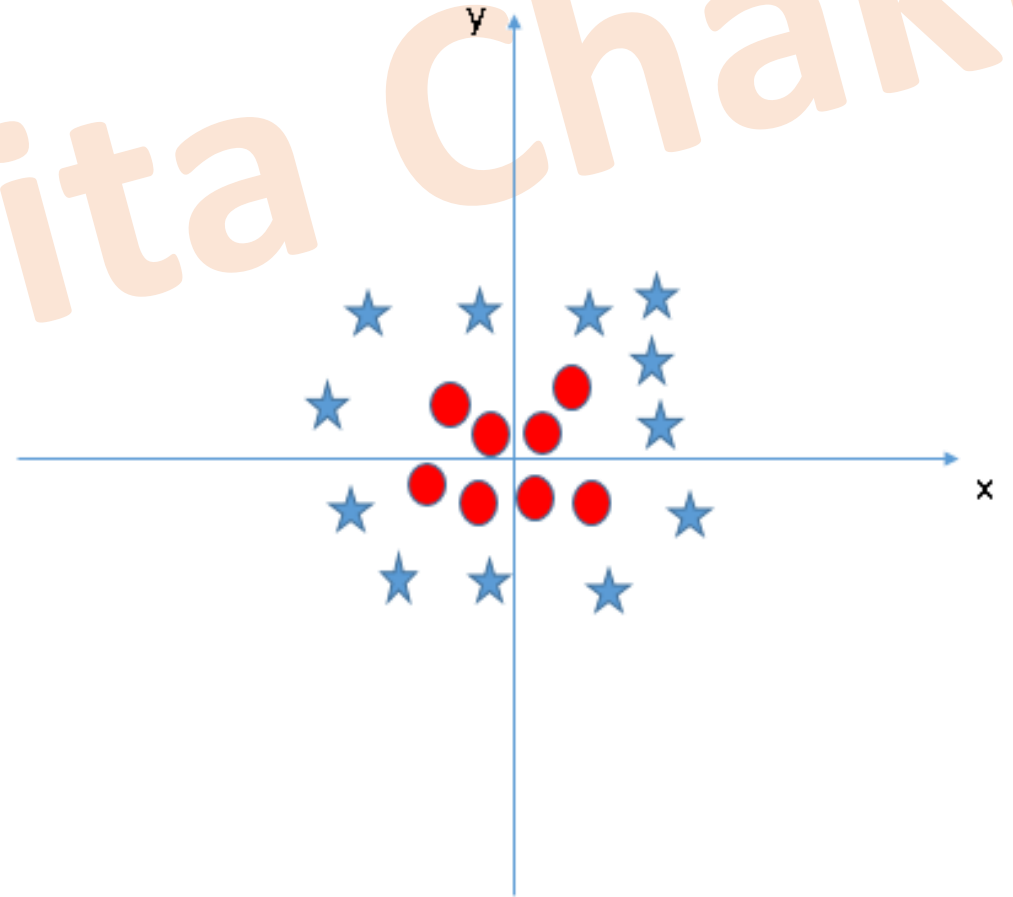
- **Can we classify two classes (Scenario-4)?:**

- As I have already mentioned, one star at other end is like an outlier for star class.
- The SVM algorithm has a feature to ignore outliers and find the hyper-plane that has the maximum margin.
- Hence, we can say, SVM classification is robust to outliers.



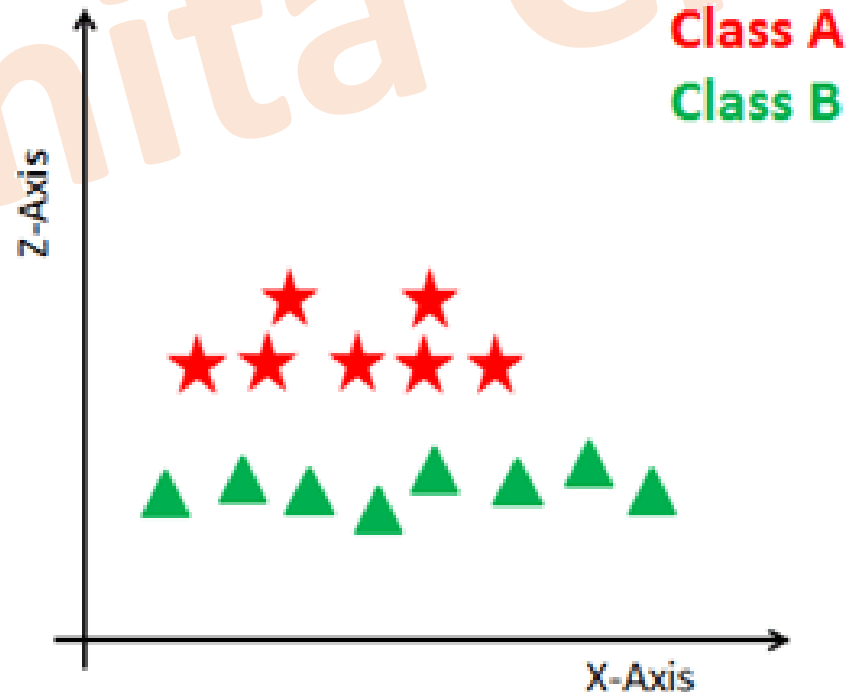
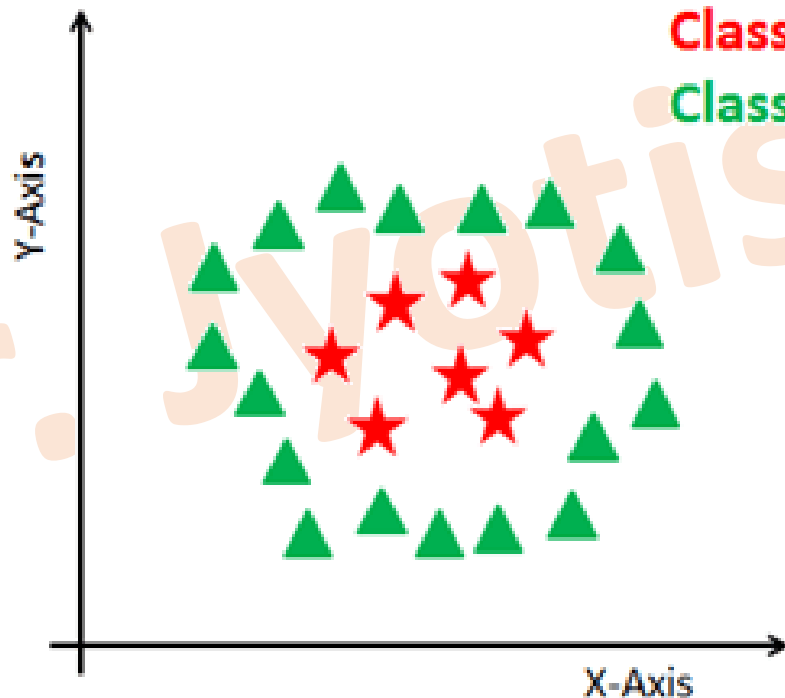
Support Vector Machine: Non Linear Hyperplane: How does it work?

- Find the hyper-plane to segregate to classes (Scenario-5):
 - In the scenario below, we can't have linear hyper-plane between the two classes, so how does SVM classify these two classes?
 - Till now, we have only looked at the **linear hyper-plane**.



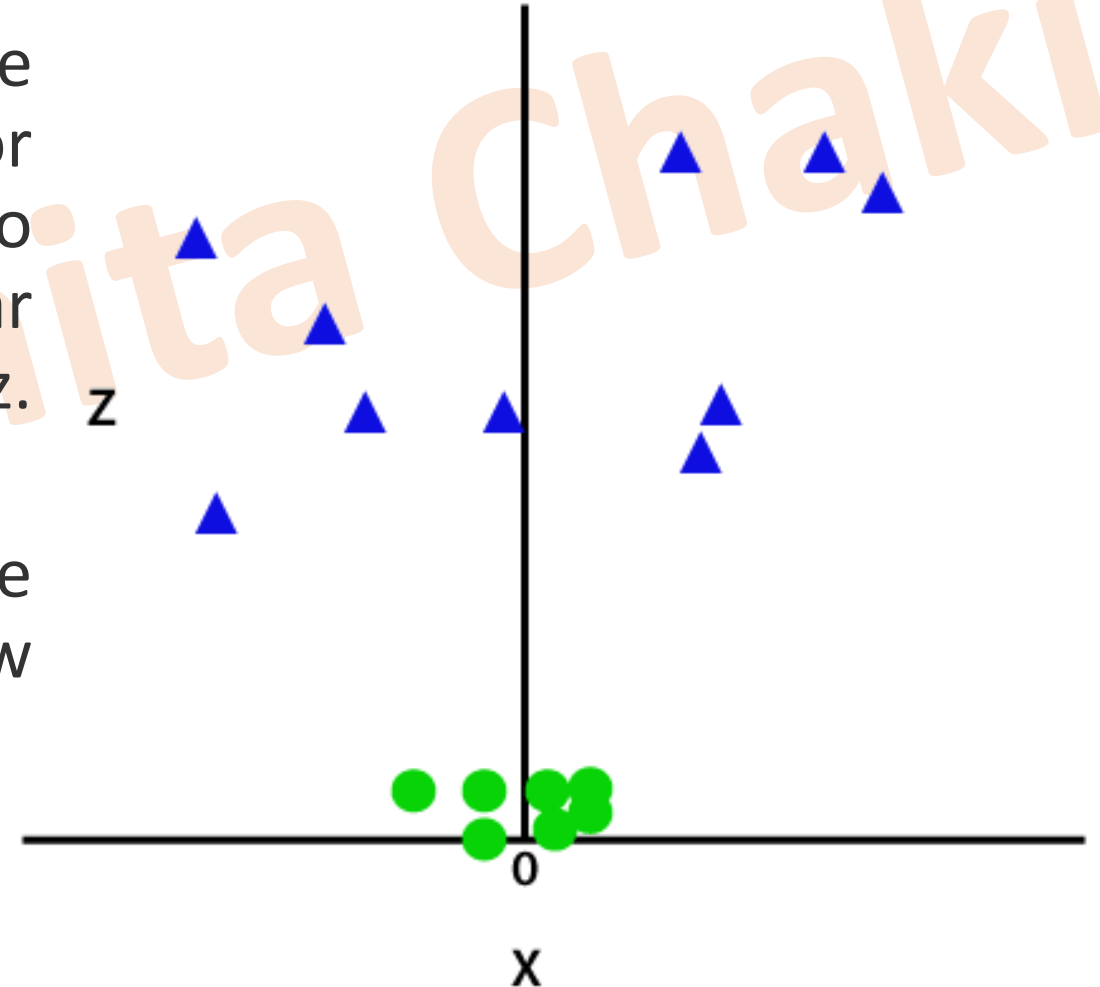
Support Vector Machine: Non Linear Hyperplane: How does it work?

- Some problems can't be solved using linear hyperplane, as shown in the figure below



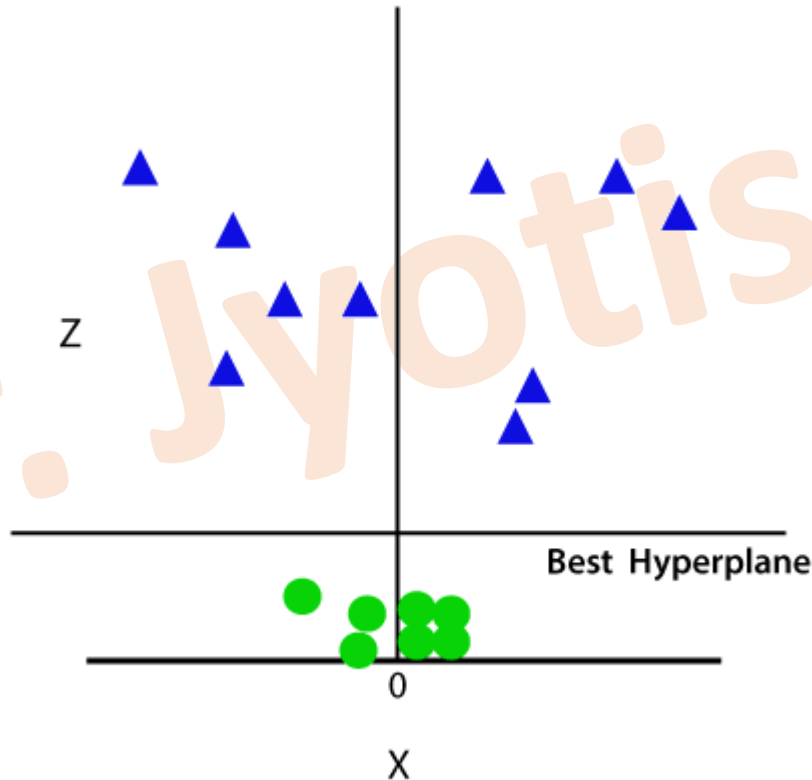
Support Vector Machine: Non Linear Hyperplane: How does it work?

- So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y , so for non-linear data, we will add a third dimension z . It can be calculated as: $z = x^2 + y^2$
- By adding the third dimension, the sample space will become as below image:



Support Vector Machine: Non Linear Hyperplane: How does it work?

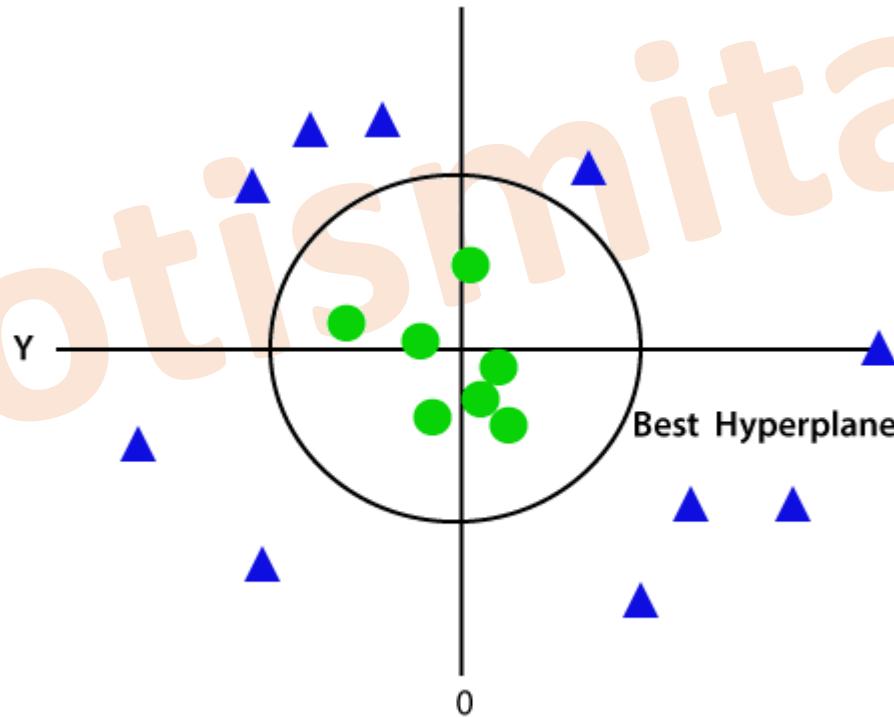
- So now, SVM will divide the datasets into classes in the following way. Consider the below image:



Since we are in 3-d Space, hence it is looking like a plane parallel to the x-axis.

Support Vector Machine: Non Linear Hyperplane: How does it work?

- If we convert it in 2d space with $z=1$, then it will become as:



- Hence we get a circumference of radius 1 in case of non-linear data.

Linear SVM with Large Margin Classification

- The fundamental idea behind SVMs is best explained with some pictures.
- **Figure 5-1** shows part of the iris dataset
- The two classes can clearly be separated easily with a straight line (they are *linearly separable*).
- The left plot shows the decision boundaries of three possible linear classifiers.
- The model whose decision boundary is represented by the dashed line is so bad that it does not even separate the classes properly.
- The other two models work perfectly on this training set, but their decision boundaries come so close to the instances that these models will probably not perform as well on new instances.
- In contrast, the solid line in the plot on the right represents the decision boundary of an SVM classifier; this line not only separates the two classes but also stays as far away from the closest training instances as possible.

Linear SVM with Large Margin Classification

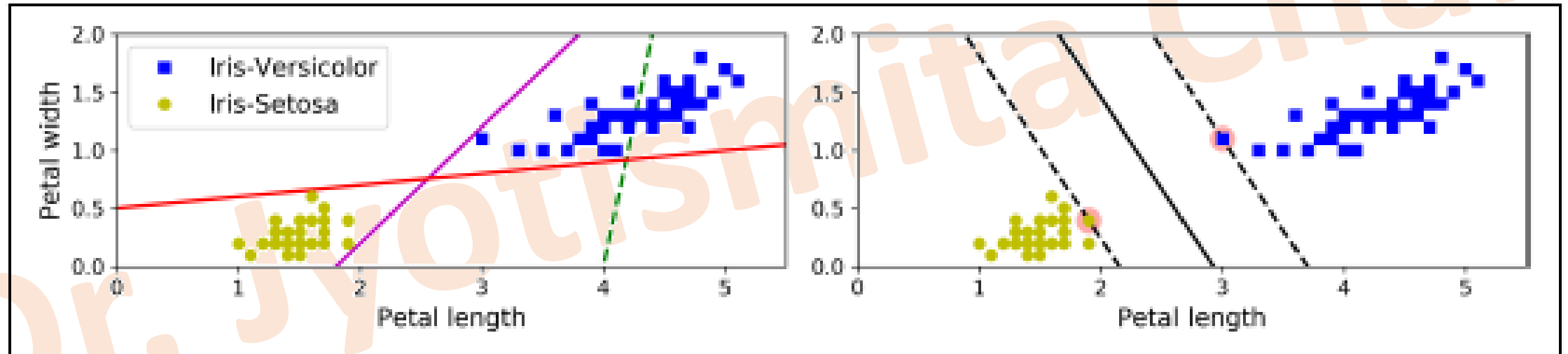


Figure 5-1. Large margin classification

Linear SVM with Large Margin Classification

- You can think of an SVM classifier as fitting the widest possible street (represented by the parallel dashed lines) between the classes.
- This is called *large margin classification*.
- Notice that adding more training instances “off the street” will not affect the decision boundary at all: it is fully determined (or “supported”) by the instances located on the edge of the street.
- These instances are called the *support vectors* (they are circled in **Figure 5-1**).

Linear SVM with Large Margin Classification

- SVMs are sensitive to the feature scales, as you can see in **Figure 5-2**: on the left plot, the vertical scale is much larger than the horizontal scale, so the widest possible street is close to horizontal.
- After feature scaling (e.g., using Scikit-Learn's StandardScaler), the decision boundary looks much better (on the right plot).

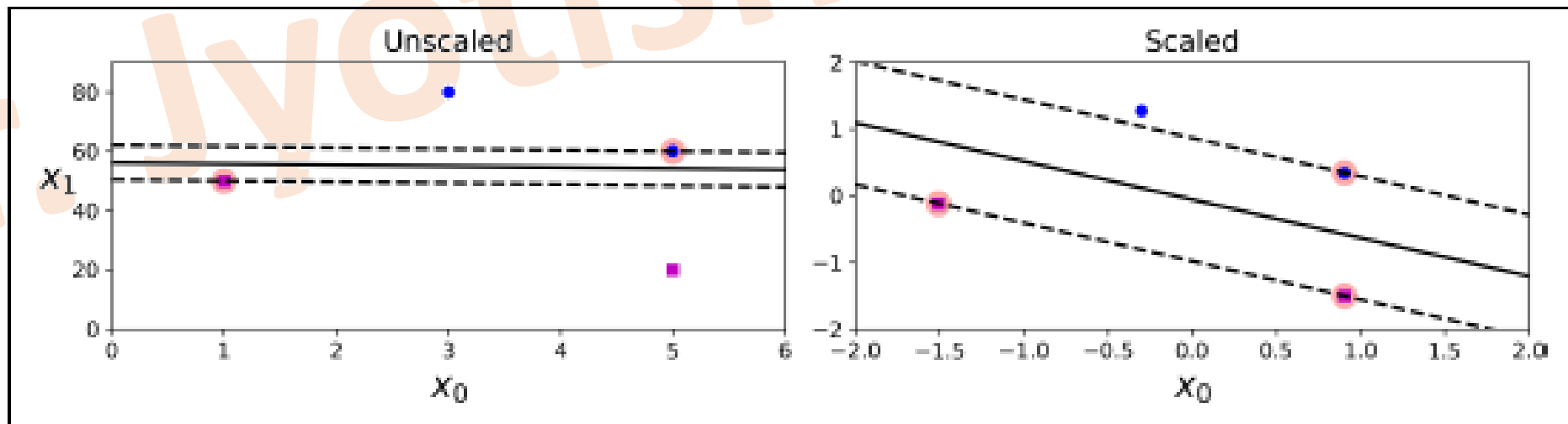


Figure 5-2. Sensitivity to feature scales

Linear SVM with Soft Margin Classification

- If we strictly impose that all instances be off the street, this is called *hard margin classification*.
- The core idea of hard margin is to maximize the margin, under the constraint that the classifier does not make any mistake.
- There are two main issues with hard margin classification.
 - First, it only works if the data is linearly separable, and second it is quite sensitive to outliers.
 - Figure 5-3 shows the iris dataset with just one additional outlier: on the left, it is impossible to find a hard margin, and on the right the decision boundary ends up very different from the one we saw in Figure 5-1 without the outlier, and it will probably not generalize as well.

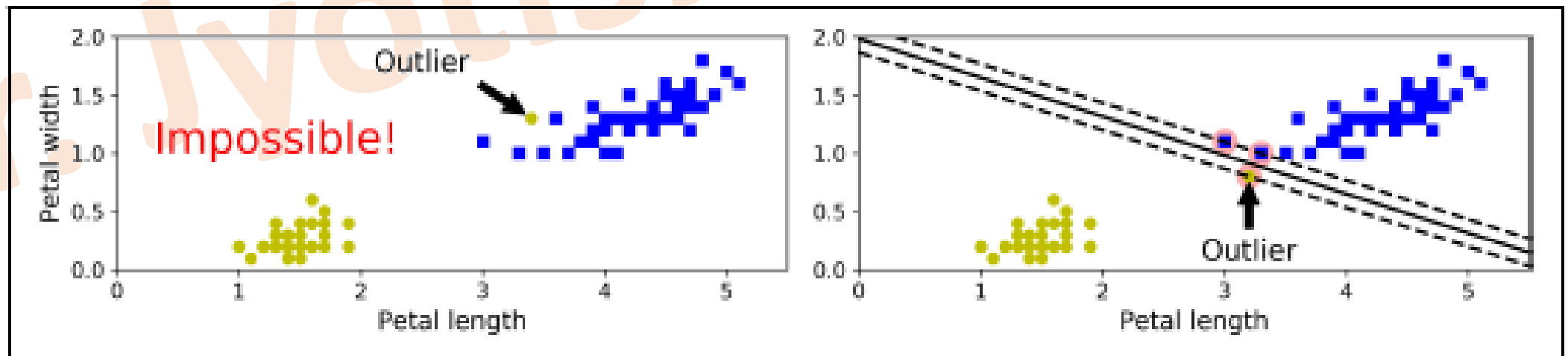
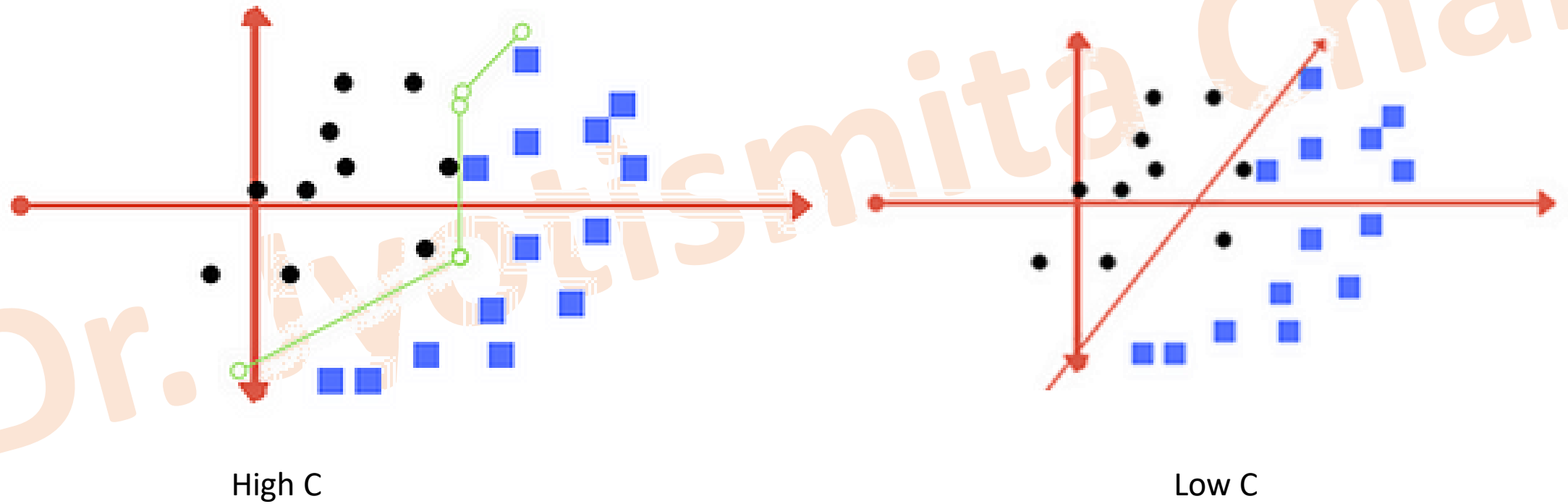


Figure 5-3. Hard margin sensitivity to outliers

Hyperparameters

- Using a high C value the classifier makes fewer margin violations but ends up with a smaller margin.
- Conversely using a low C value the margin is much larger, but many instances end up on the hyperplane.
- If the SVM model is overfitting you can use C hyper-parameter to regularize it.
- With a low C , the decision boundary is smooth and potentially simpler, and some misclassifications are tolerated.
- With a high C , the decision boundary is more complex, trying to correctly classify more points.

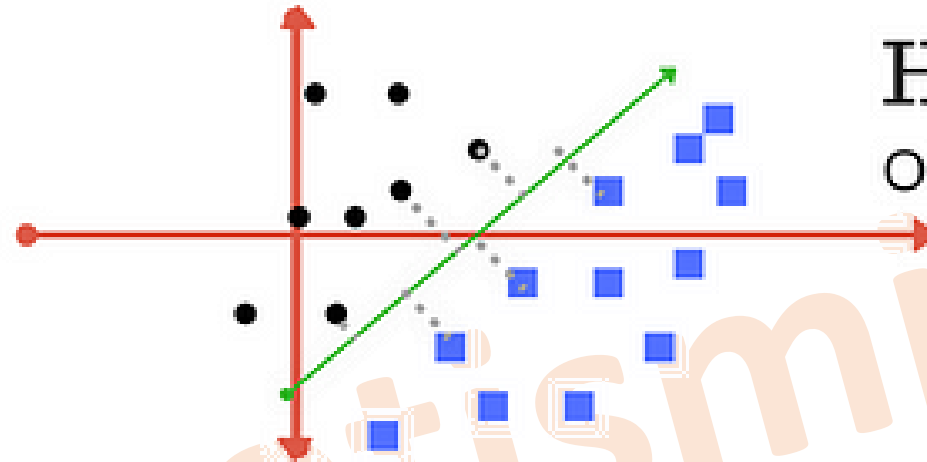
Hyperparameters



Hyperparameters

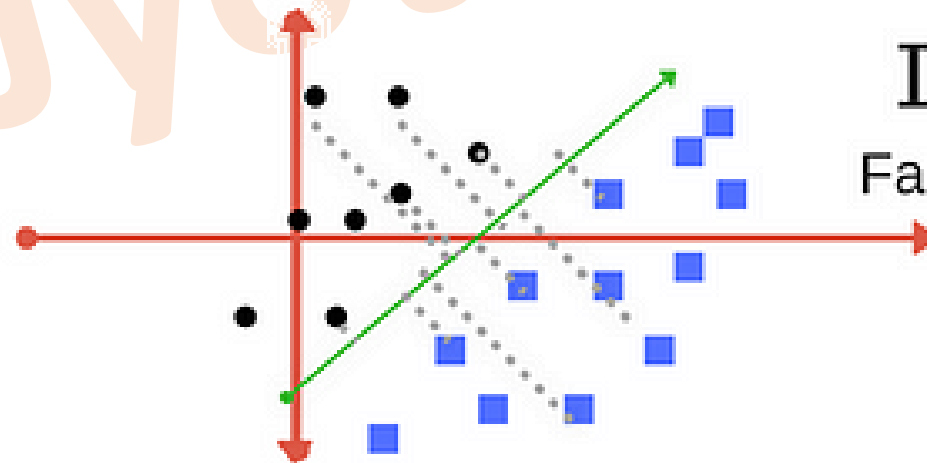
- The **gamma parameter** is the inverse of the standard deviation of the RBF kernel (Gaussian function), which is used as a similarity measure between two points.
- Intuitively, a small gamma value defines a Gaussian function with a large variance.
- In this case, two points can be considered similar even if they are far from each other.
- On the other hand, a large gamma value means define a Gaussian function with a small variance and in this case, two points are considered similar just if they are close to each other.
- With a low gamma, the decision boundary does not bend to with respect to individual points, leading to a more general classification.
- With a high gamma, the decision boundary bends and twists to classify the training points correctly, which may capture noise and lead to overfitting.

Hyperparameters



High Gamma

Only nearby points are considered.



Low Gamma

Far away points are also considered.

Linear SVM with Soft Margin Classification

- To avoid the issues related to hard margin it is preferable to use a more flexible model.
- The objective is to find a good balance between keeping the street as large as possible and limiting the *margin violations* (i.e., instances that end up in the middle of the street or even on the wrong side).
- This is called *soft margin classification*.
- In Scikit-Learn's SVM classes, you can control this balance using the C hyperparameter: a smaller C value leads to a wider street but more margin violations.
- Figure 5-4 shows the decision boundaries and margins of two soft margin SVM classifiers on a nonlinearly separable dataset.
- On the left, using a low C value the margin is quite large, but many instances end up on the street.
- On the right, using a high C value the classifier makes fewer margin violations but ends up with a smaller margin.
- However, it seems likely that the first classifier will generalize better: in fact even on this training set it makes fewer prediction errors, since most of the margin violations are actually on the correct side of the decision boundary.
- If your SVM model is overfitting, you can try regularizing it by reducing C.

Linear SVM with Soft Margin Classification

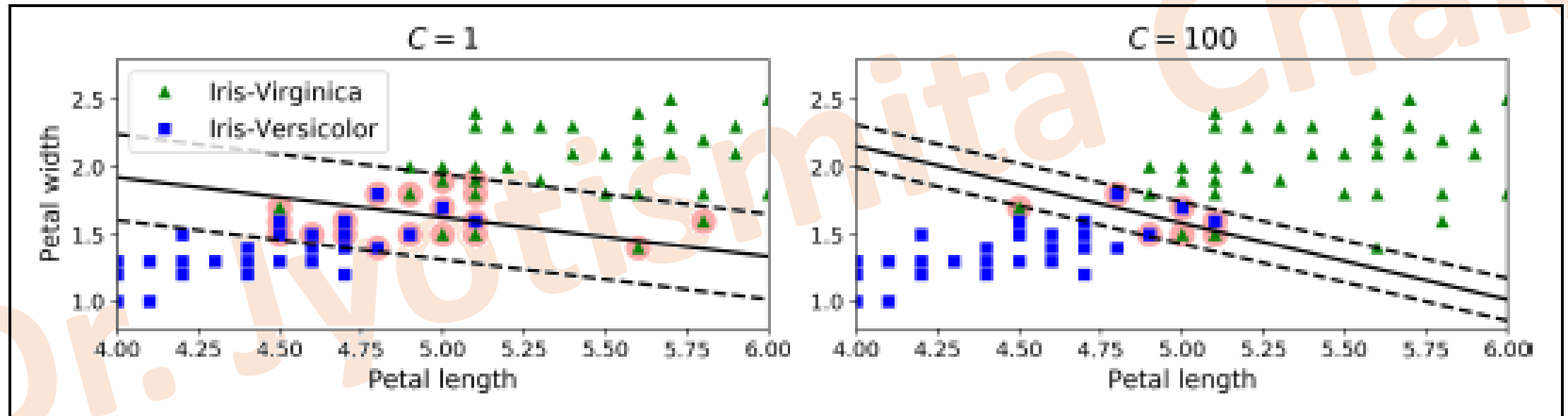


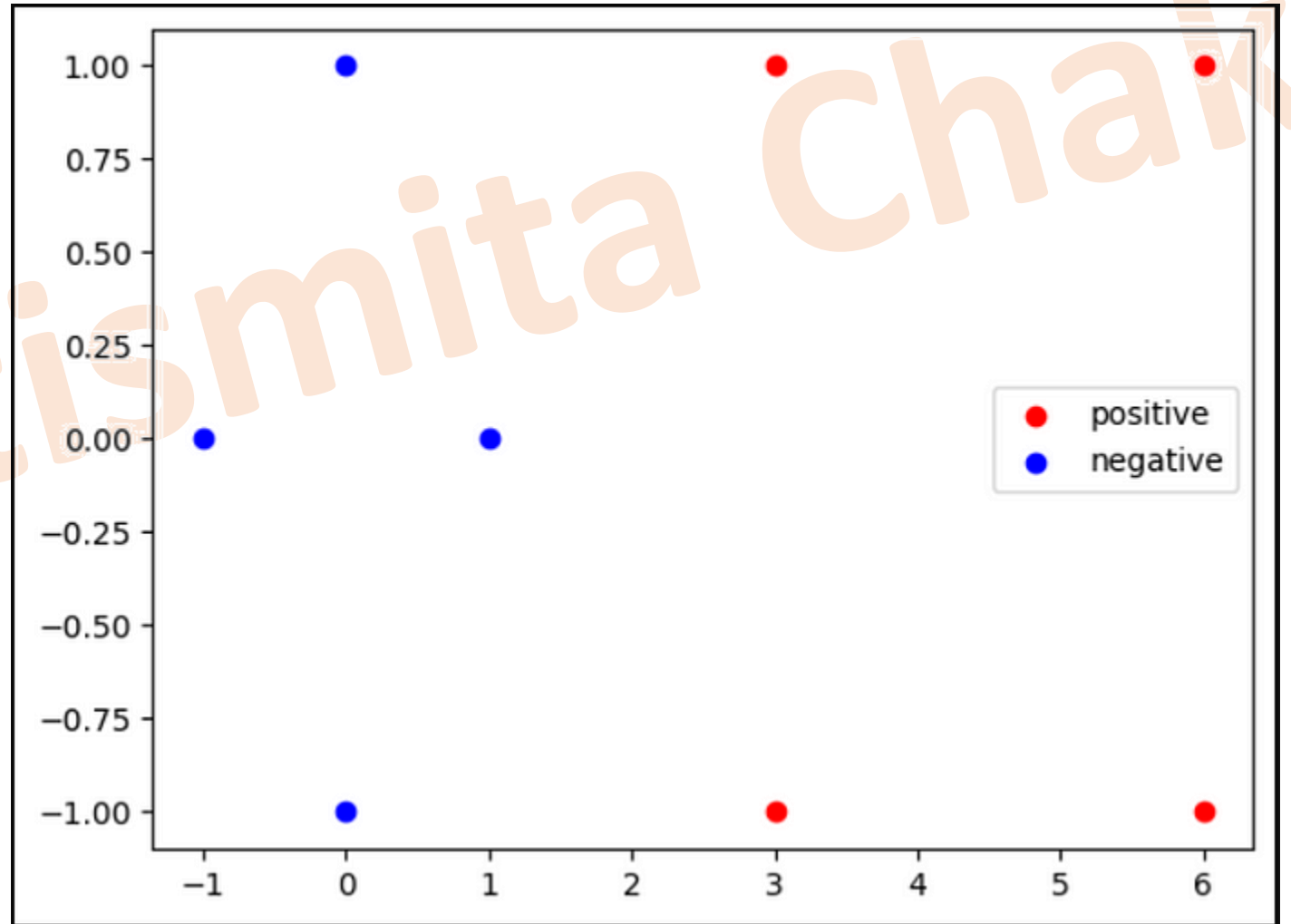
Figure 5-4. Large margin (left) versus fewer margin violations (right)

Linear SVM: Example

- Positively labelled data points $(3,1)(3,-1)(6,1)(6,-1)$ and Negatively labelled data points $(1,0)(0,1)(0,-1)(-1,0)$
- Draw the hyperplane which will separate positive and negative data points by using SVM.

Linear SVM: Example

- For all negative labelled output is -1 and for all positive labelled output is 1.
- In this case the SVs are as follows:
 - Negative: (1, 0)
 - Positive: (3, 1), (3, -1)



Linear SVM: Example

- Now adding 1 (bias) to all points:

- $s_1 = (3,1) \Rightarrow s_1' = (3,1,1)$
- $s_2 = (3,-1) \Rightarrow s_2' = (3,1,-1)$
- $s_3 = (6,1) \Rightarrow s_3' = (6,1,1)$
- $s_4 = (6,-1) \Rightarrow s_4' = (6,1,-1)$
- $s_5 = (1,0) \Rightarrow s_5' = (1,0,1)$
- $s_6 = (0,1) \Rightarrow s_6' = (0,1,1)$
- $s_7 = (0,-1) \Rightarrow s_7' = (0,-1,1)$
- $s_8 = (-1,0) \Rightarrow s_8' = (-1,0,1)$

We can see there is one negative point $s_1 = (1,0,1)$ and two positive points $s_2 = (3,1,1)$ & $s_3 = (3,-1,1)$ form support vectors

Linear SVM: Example

- Generalized equation

- $\alpha_1 * s_1' * s_1' + \alpha_2 * s_1' * s_2' + \alpha_3 * s_1' * s_3' = -1$ (1)

- $\alpha_1 * s_2' * s_1' + \alpha_2 * s_2' * s_2' + \alpha_3 * s_2' * s_3' = 1$ (2)

- $\alpha_1 * s_3' * s_1' + \alpha_2 * s_3' * s_2' + \alpha_3 * s_3' * s_3' = 1$ (3)

- On solving these equations

- eq 1 $\Rightarrow \alpha_1 * (1,0,1) * (1,0,1) + \alpha_2 * (1,0,1) * (3,1,1) + \alpha_3 * (1,0,1) * (3,-1,1) = -1$

- eq 2 $\Rightarrow \alpha_1 * (3,1,1) * (1,0,1) + \alpha_2 * (3,1,1) * (3,1,1) + \alpha_3 * (3,1,1) * (3,-1,1) = 1$

- eq 3 $\Rightarrow \alpha_1 * (3,-1,1) * (1,0,1) + \alpha_2 * (3,-1,1) * (3,1,1) + \alpha_3 * (3,-1,1) * (3,-1,1) = 1$

- $2\alpha_1 + 4\alpha_2 + 4\alpha_3 = -1$ (4)

- $4\alpha_1 + 11\alpha_2 + 9\alpha_3 = 1$ (5)

- $4\alpha_1 + 9\alpha_2 + 11\alpha_3 = 1$ (6)

Linear SVM: Example

- On solving these equations 4,5 and 6 we get,
 - $\alpha_1 = -3.5$, $\alpha_2 = 0.75$ and $\alpha_3 = 0.75$

- To find hyper-plane

- $W' = \sum \alpha_i * s_i$
- $W' = -3.5 * (1,0,1) + 0.75 * (3,1,1) + 0.75 * (3,-1,1)$
- $W' = (1,0,-2)$
- $y = W'x + b$
- $W' = (1,0)$ and $b = -2$

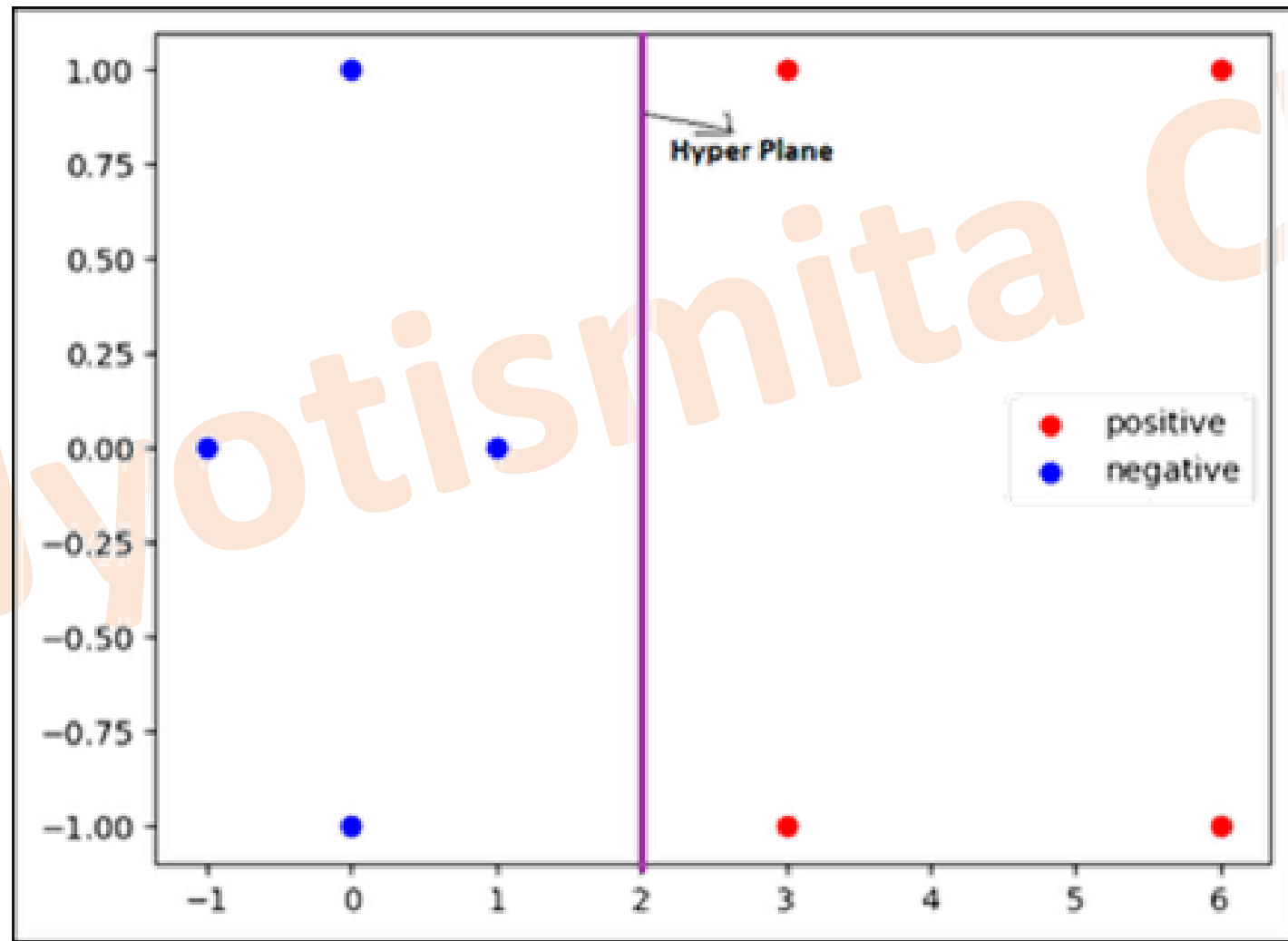
- So the best fit line or hyper plane is at $(0,2)$ which splits the data points into two classes

$$\bar{w} = \begin{pmatrix} 1 \\ 0 \\ -2 \end{pmatrix}$$



$\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ line is parallel to x_2 axis	
$\begin{pmatrix} 0 \\ 1 \end{pmatrix}$ line is parallel to x_1 axis	
$\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ line will make 45° with respect to the x_1 and x_2 axis	

Linear SVM: Example

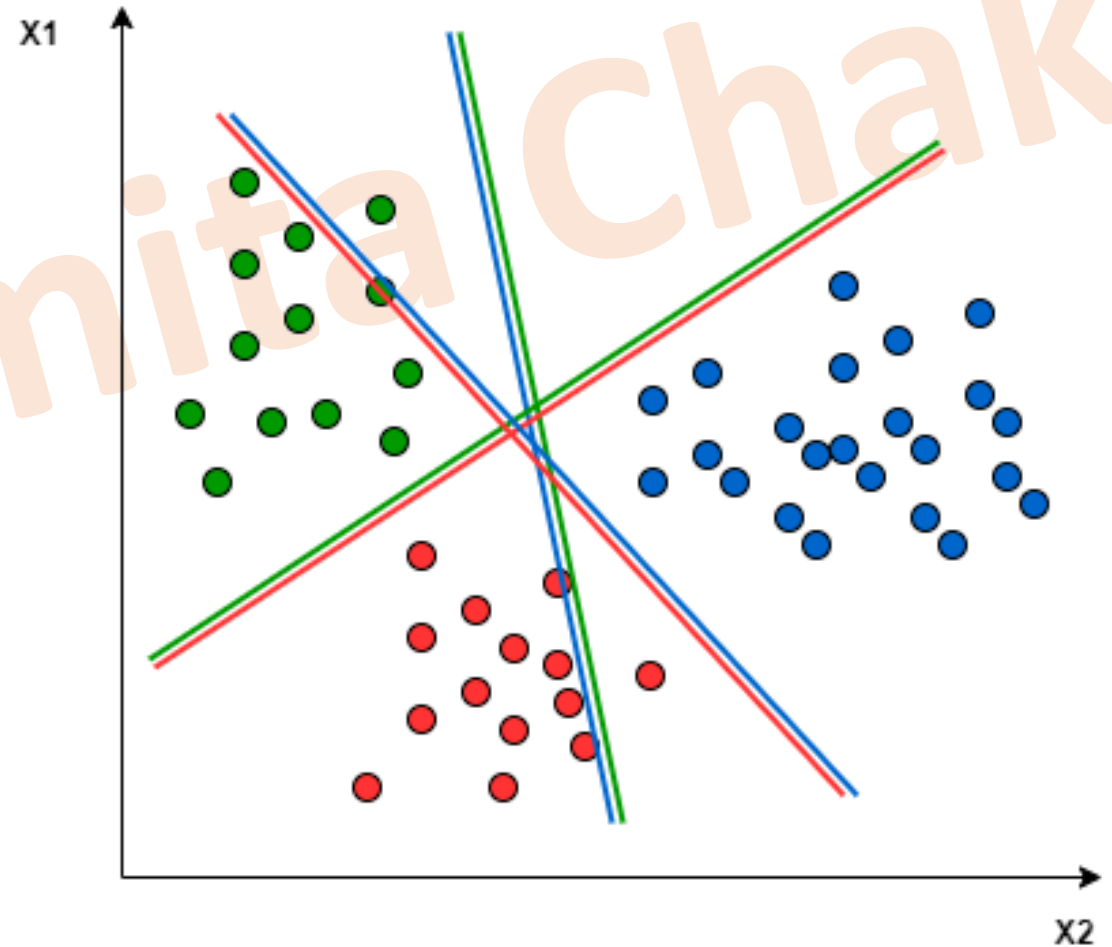


SVM: Multiclass classification

- SVM doesn't support multiclass classification natively.
- It supports binary classification and separating data points into two classes.
- For multiclass classification, the same principle is utilized after breaking down the multiclassification problem into multiple binary classification problems.
- The idea is to map data points to high dimensional space to gain mutual linear separation between every two classes.
- This is called a **One-to-One approach**, which breaks down the multiclass problem into multiple binary classification problems. A binary classifier per each pair of classes.
- Another approach one can use is **One-to-Rest**. In that approach, the breakdown is set to a binary classifier per each class.

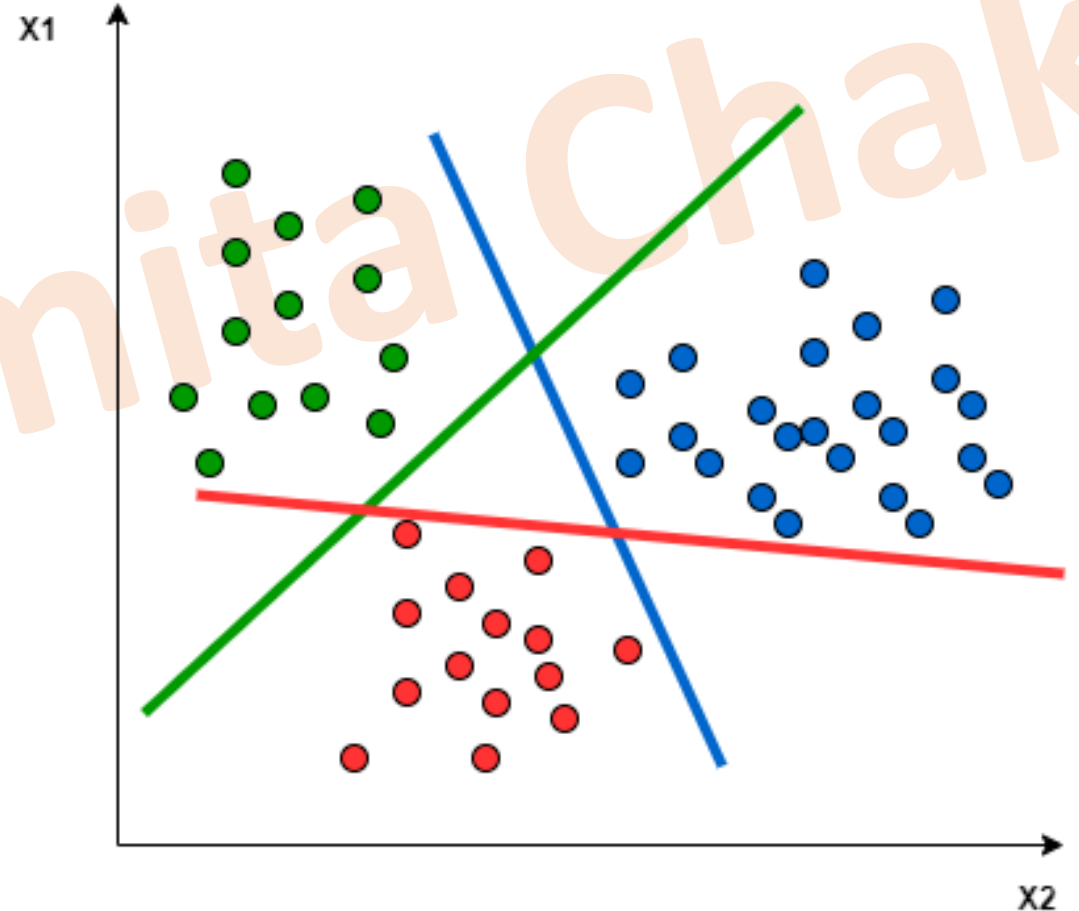
SVM: Multiclass classification

- In the *One-to-One* approach, we need a hyperplane to separate between every two classes, neglecting the points of the third class.
- This means the separation takes into account only the points of the two classes in the current split.
- For example, the red-blue line tries to maximize the separation only between blue and red points. It has nothing to do with green points:



SVM: Multiclass classification

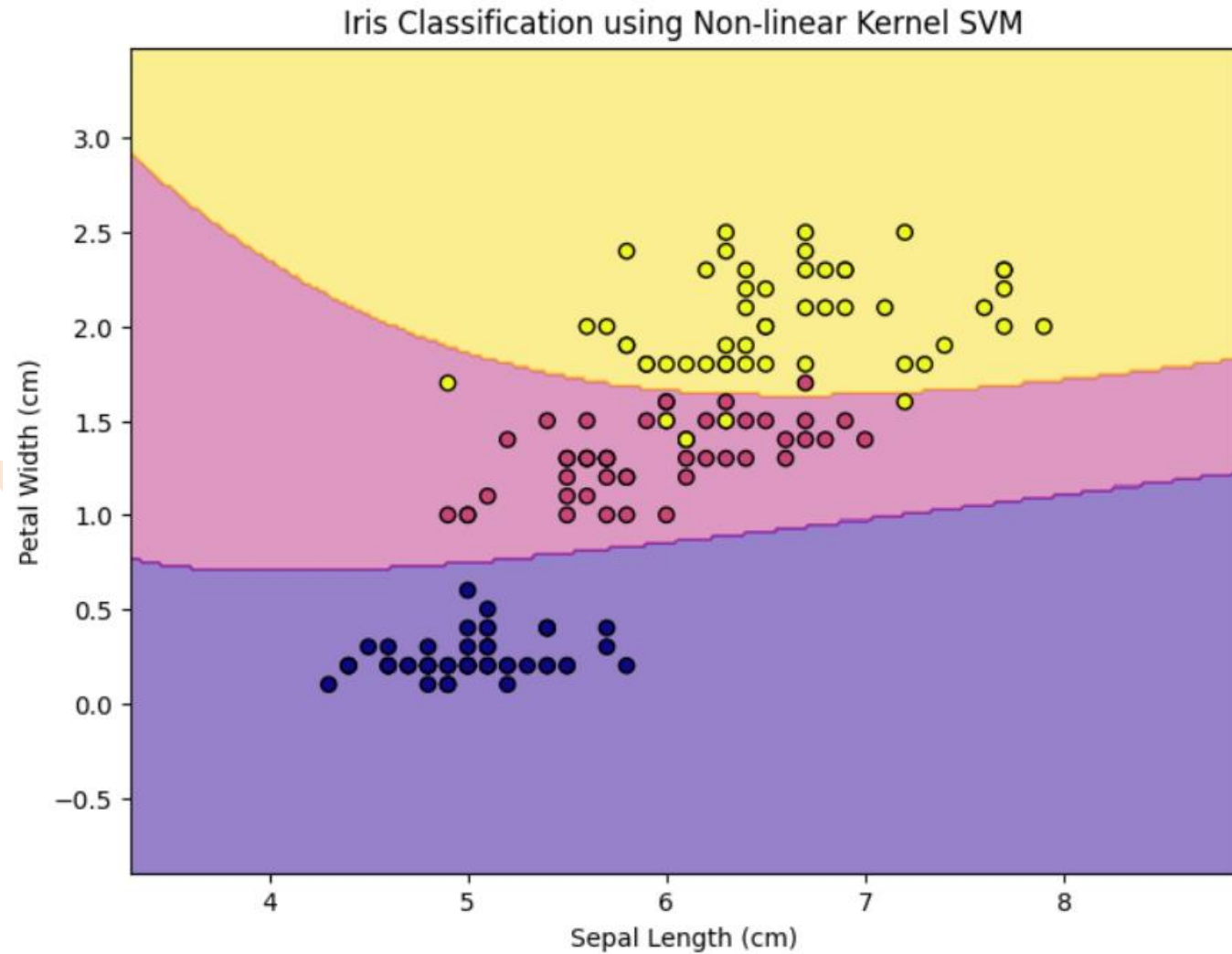
- In the *One-to-Rest* approach, we need a hyperplane to separate between a class and all others at once.
- This means the separation takes all points into account, dividing them into two groups; a group for the class points and a group for all other points.
- For example, the green line tries to maximize the separation between green points and all other points at once:



Non-linear SVM Classification

- Nonlinear SVM is necessary when the data cannot be effectively separated by a linear decision boundary in the original feature space.
- Nonlinear SVM addresses this limitation by utilizing kernel functions to map the data into a higher-dimensional space where linear separation becomes possible.
- The kernel function computes the similarity between data points, allowing SVM to capture complex patterns and nonlinear relationships between features.
- This enables nonlinear SVM to handle intricate data distributions, such as curved or circular decision boundaries.
- By leveraging the kernel trick, nonlinear SVM provides a powerful tool for solving classification problems where linear separation is insufficient, extending its applicability to a wide range of real-world scenarios.

Non-linear SVM Classification



Non-linear SVM Classification: Polynomial kernel

- Polynomial kernel SVM is a type of SVM that uses a polynomial function to transform the input data into a higher dimensional space.
- The polynomial kernel function takes the dot product of the input data points and adds a constant to the result, which is raised to a power specified by the degree parameter of the function. $F(x, x_j) = (x \cdot x_j + 1)^d$
- The result of this transformation is a set of new features that capture the non-linear relationships between the input data.
- The performance of polynomial kernel SVM depends on the choice of its parameters, such as the degree of the polynomial function and the regularization parameter.
- Choosing the right parameters is crucial for achieving good performance on the test data.
- There are several techniques for choosing the parameters, such as grid search and cross-validation.

Non-linear SVM Classification: Polynomial kernel

```
from sklearn.datasets import make_moons
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures

polynomial_svm_clf = Pipeline([
    ("poly_features", PolynomialFeatures(degree=3)),
    ("scaler", StandardScaler()),
    ("svm_clf", LinearSVC(C=10, loss="hinge"))
])

polynomial_svm_clf.fit(X, y)
```

Non-linear SVM Classification: Polynomial kernel

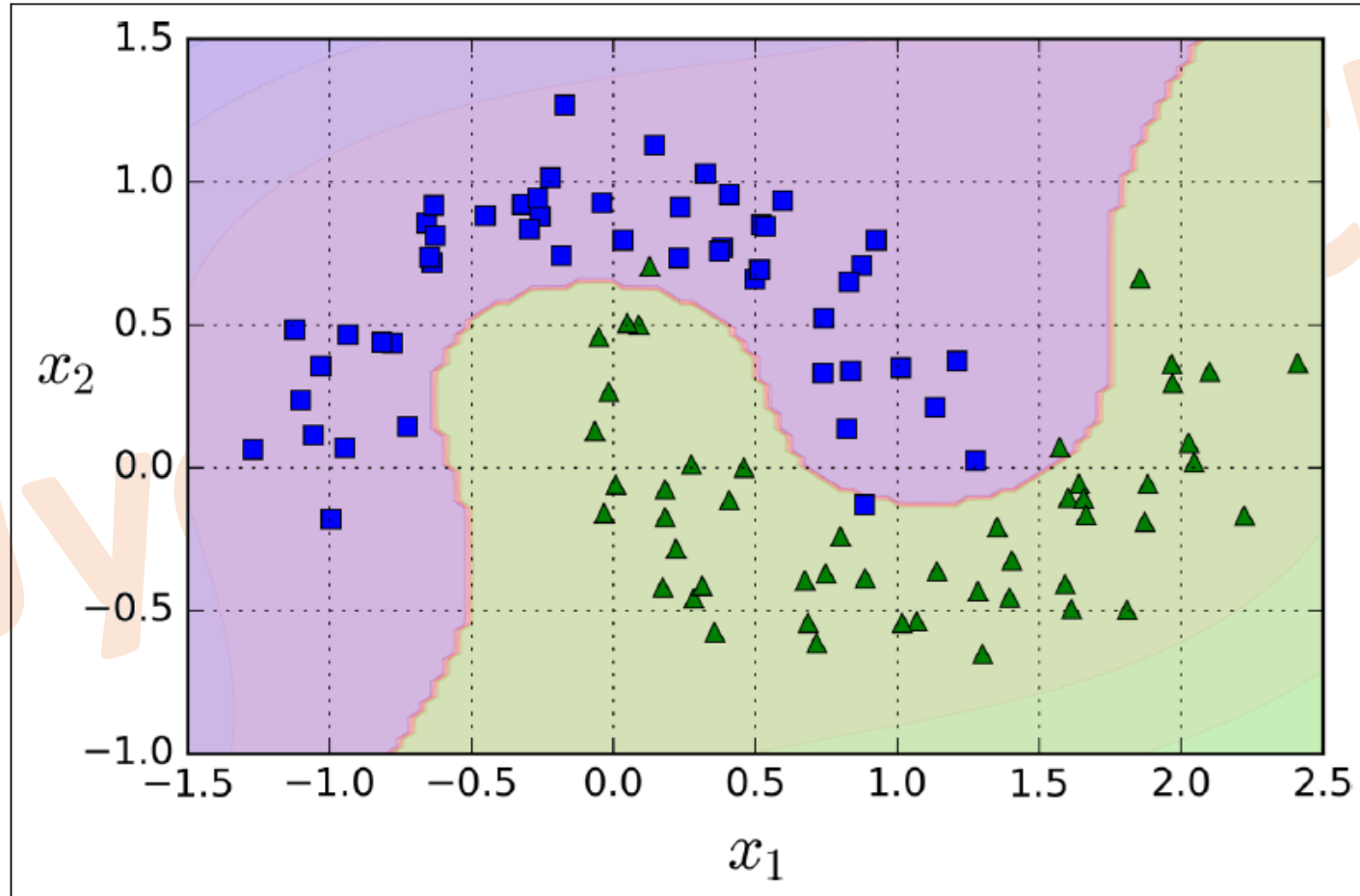


Figure 5-6. Linear SVM classifier using polynomial features

Non-linear SVM Classification: Polynomial kernel

- In the polynomial kernel function, `coef0` is a constant term that is added to the dot product of the input features before raising it to the power of the degree.
- It controls the position of the decision boundary.
- A positive value of `coef0` shifts the decision boundary to the positive side and a negative value of `coef0` shifts the decision boundary to the negative side.

```
from sklearn.svm import SVC
poly_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5))
])
poly_kernel_svm_clf.fit(X, y)
```

Non-linear SVM Classification: Polynomial kernel

- This code trains an SVM classifier using a 3rd-degree polynomial kernel.
- It is represented on the left of **Figure 5-7**.
- On the right is another SVM classifier using a 10th degree polynomial kernel. Obviously, if your model is overfitting, you might want to reduce the polynomial degree.
- Conversely, if it is underfitting, you can try increasing it.
- A common approach to find the right hyperparameter values is to use grid search.
- It is often faster to first do a very coarse grid search, then a finer grid search around the best values found.
- Having a good sense of what each hyperparameter actually does can also help you search in the right part of the hyperparameter space.

Non-linear SVM Classification: Polynomial kernel

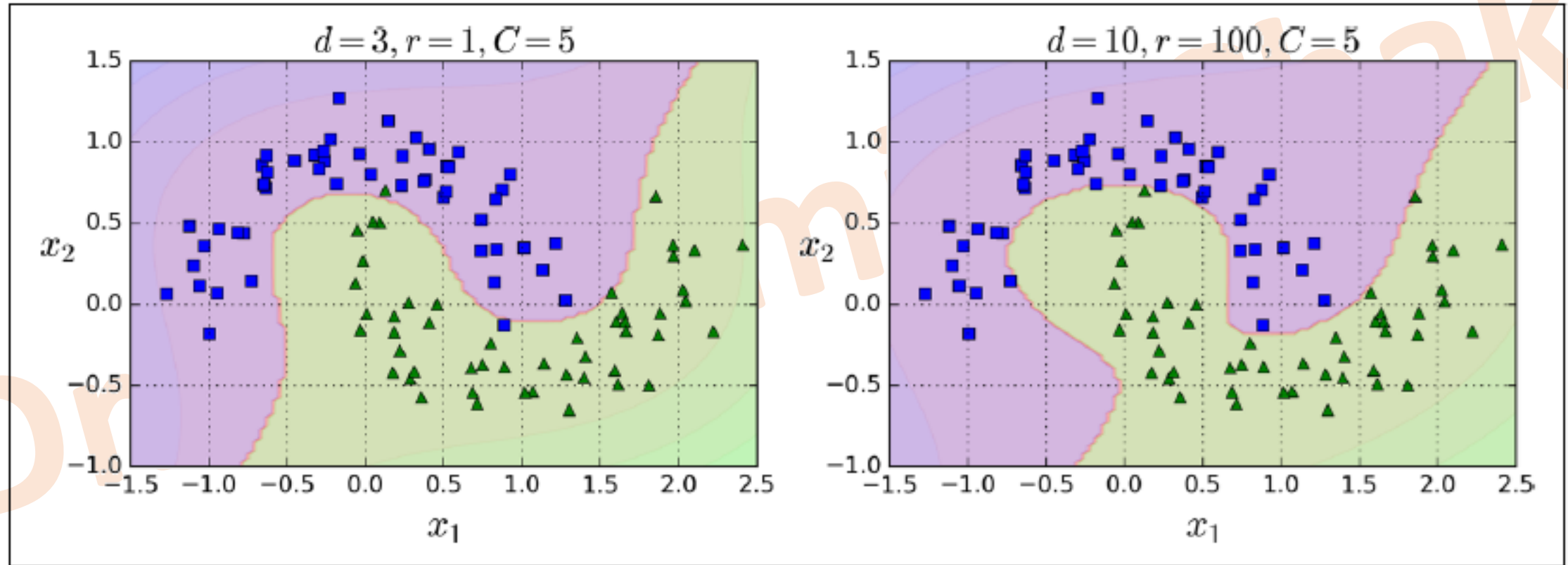
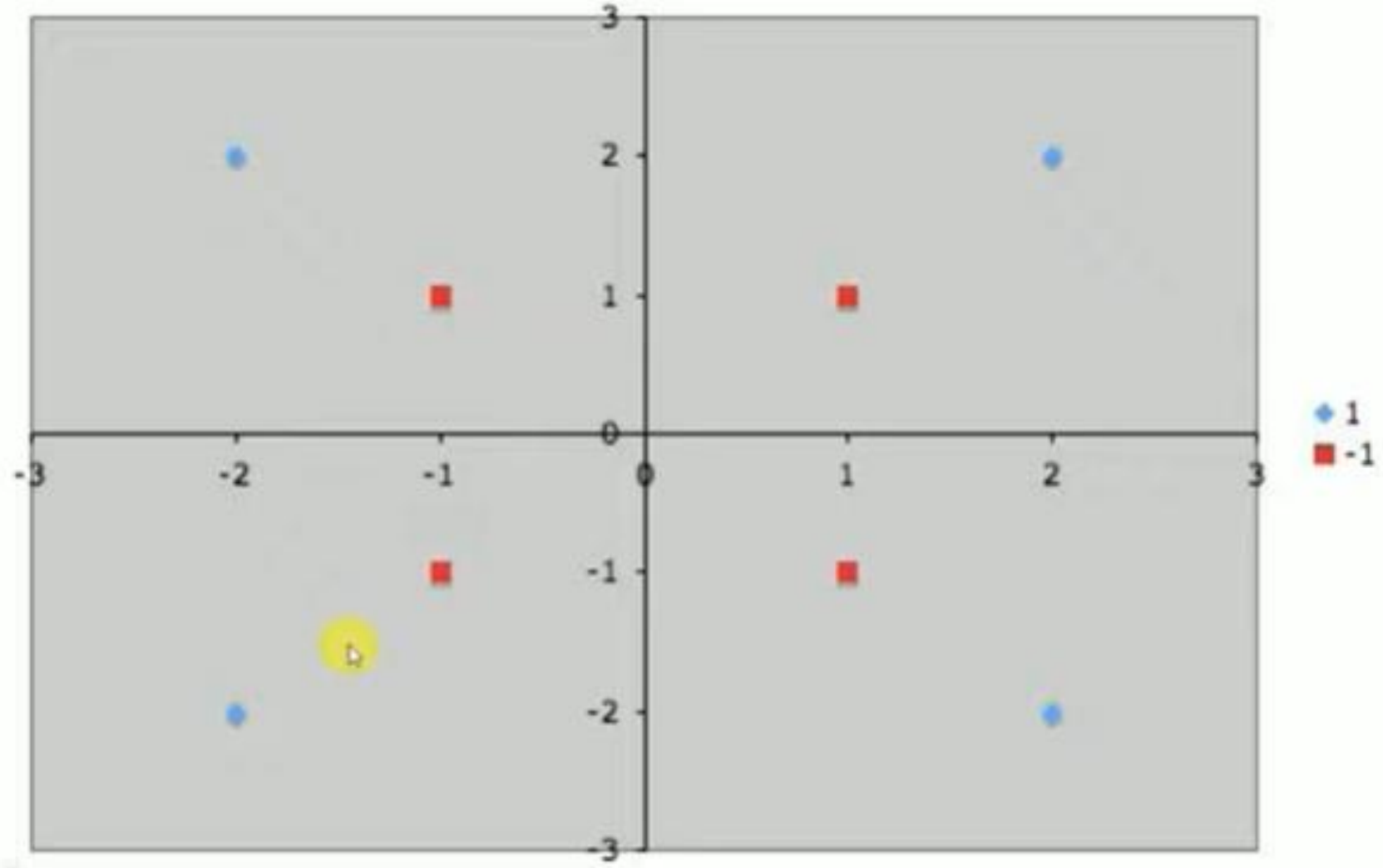


Figure 5-7. SVM classifiers with a polynomial kernel

Non-linear SVM: Example

- Positively labelled data points $(2,2)(2,-2)(-2,-2)(-2,2)$ and Negatively labelled data points $(1,1)(1,-1)(-1,-1)(-1,1)$.
- Draw the hyperplane which will separate positive and negative data points by using SVM.

Non-linear SVM: Example



Non-linear SVM: Example

- We have to find a hyper-plane i.e it divides data into two different classes.
- But this is of type non-linear SVM so we need to convert data from one feature space to another feature space using Kernel SVM.
- Kernel SVM Condition:

$$\Phi_1 \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{cases} \begin{pmatrix} 4 - x_2 + |x_1 - x_2| \\ 4 - x_1 + |x_1 - x_2| \end{pmatrix} & \text{if } \sqrt{x_1^2 + x_2^2} > 2 \\ \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} & \text{otherwise} \end{cases}$$

Non-linear SVM: Example

- For positive labelled data:

- $\theta(2,2) \Rightarrow (\sqrt{2^2 + 2^2}) = (\sqrt{8}) > 2$

- so $\theta(x_1) = 4 - 2 + |2 - 2| \Rightarrow \theta(x_1) = 2$

- $\theta(x_2) = 4 - 2 + |2 - 2| \Rightarrow \theta(x_2) = 2$

- $\theta(2,-2) \Rightarrow (\sqrt{2^2 + 2^2}) = (\sqrt{8}) > 2$

- so $\theta(x_1) = 4 + 2 + |2 + 2| \Rightarrow \theta(x_1) = 10$

- $\theta(x_2) = 4 - 2 + |2 + 2| \Rightarrow \theta(x_2) = 6$

- $\theta(-2,-2) \Rightarrow (\sqrt{2^2 + 2^2}) = (\sqrt{8}) > 2$

- so $\theta(x_1) = 4 + 2 + |-2 + 2| \Rightarrow \theta(x_1) = 6$

- $\theta(x_2) = 4 + 2 + |-2 + 2| \Rightarrow \theta(x_2) = 6$

- $\theta(-2,2) \Rightarrow (\sqrt{2^2 + 2^2}) = (\sqrt{8}) > 2$

- so $\theta(x_1) = 4 - 2 + |-2 - 2| \Rightarrow \theta(x_1) = 6$

- $\theta(x_2) = 4 + 2 + |-2 - 2| \Rightarrow \theta(x_2) = 10$

- Hence overall positive labelled data is rechanged as (2,2), (10,6), (6,6), (6,10)

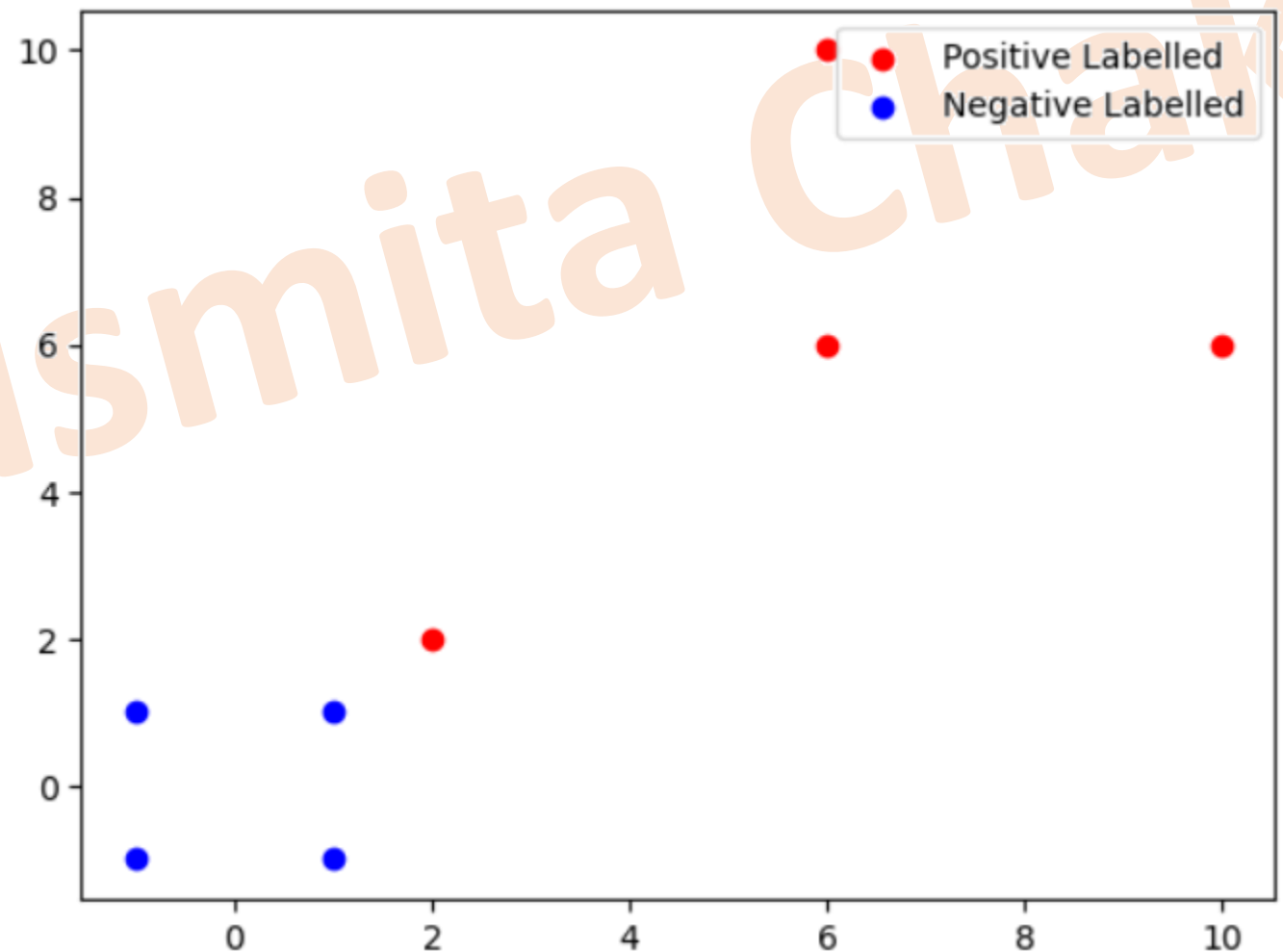
Non-linear SVM: Example

- For negative labelled data points:

- Here all values are (1, 1) only thing is negative and positive sign changes, so if we square root them then all values will be less than 2.

- Finally there is no change in the points.

- Final plot is →



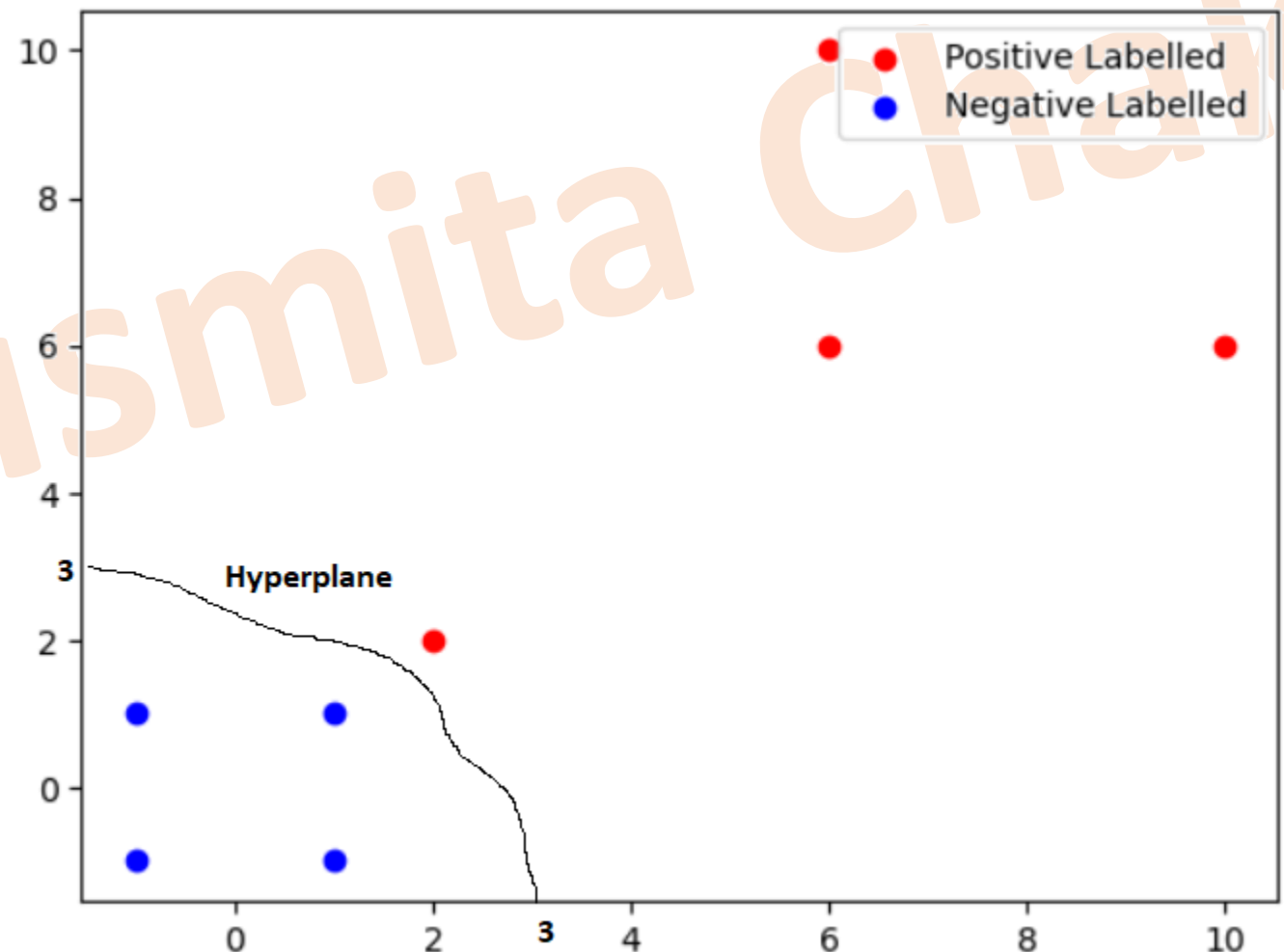
Non-linear SVM: Example

- From the graph we can say support vectors are $S_1 = (1,1)$ and $S_2 = (2,2)$ so adding 1 to support vectors, $S_1' = (1,1,1)$ and $S_2' = (2,2,1)$.
- $\alpha_1 * S_1' * S_1' + \alpha_2 * S_1' * S_2' = -1$ (1)
- $\alpha_1 * S_2' * S_1' + \alpha_2 * S_2' * S_2' = 1$ (2)
- $\alpha_1 * (1,1,1) * (1,1,1) + \alpha_2 * (1,1,1) * (2,2,1) = -1$
- $\alpha_1 * (2,2,1) * (1,1,1) + \alpha_2 * (2,2,1) * (2,2,1) = 1$
- $3\alpha_1 + 5\alpha_2 = -1$ and $5\alpha_1 + 9\alpha_2 = 1$
- On solving equations $\alpha_1 = -7$ and $\alpha_2 = 4$

Non-linear SVM: Example

- To find hyper-plane

- $W' = \sum \alpha_i * s_i$
- $W' = -7 * (1,1,1) + 4 * (2,2,1)$
- $W' = (1,1,-3)$
- $y = W'x + b$
- $W' = 1,1$ and $b = -3$



Similarity features

- Another technique to tackle nonlinear problems is to add features computed using a *similarity function* that measures how much each instance resembles a particular *landmark*.
- For example, let's take the one-dimensional dataset and add two landmarks to it at $x_1 = -2$ and $x_1 = 1$ (see the left plot in [Figure 5-8](#)).
- Next, let's define the similarity function to be the Gaussian *Radial Basis Function (RBF)* with $\gamma = 0.3$. ℓ is the landmark.

$$\phi_{\gamma}(\mathbf{x}, \ell) = \exp \left(-\gamma \| \mathbf{x} - \ell \|^2 \right)$$

- It is a bell-shaped function varying from 0 (very far away from the landmark) to 1 (at the landmark).
- Now we are ready to compute the new features.

Similarity features

- For example, let's look at the instance $x_1 = -1$: it is located at a distance of 1 from the first landmark, and 2 from the second landmark.
- Therefore its new features are $x_2 = \exp(-0.3 \times 1^2) \approx 0.74$ and $x_3 = \exp(-0.3 \times 2^2) \approx 0.30$.
- The plot on the right of **Figure 5-8** shows the transformed dataset (dropping the original features).
- As you can see, it is now linearly separable.
- The simplest approach is to create a landmark at the location of each and every instance in the dataset.
- This creates many dimensions and thus increases the chances that the transformed training set will be linearly separable.
- The downside is that a training set with m instances and n features gets transformed into a training set with m instances and m features (assuming you drop the original features).
- If your training set is very large, you end up with an equally large number of features.

Similarity features

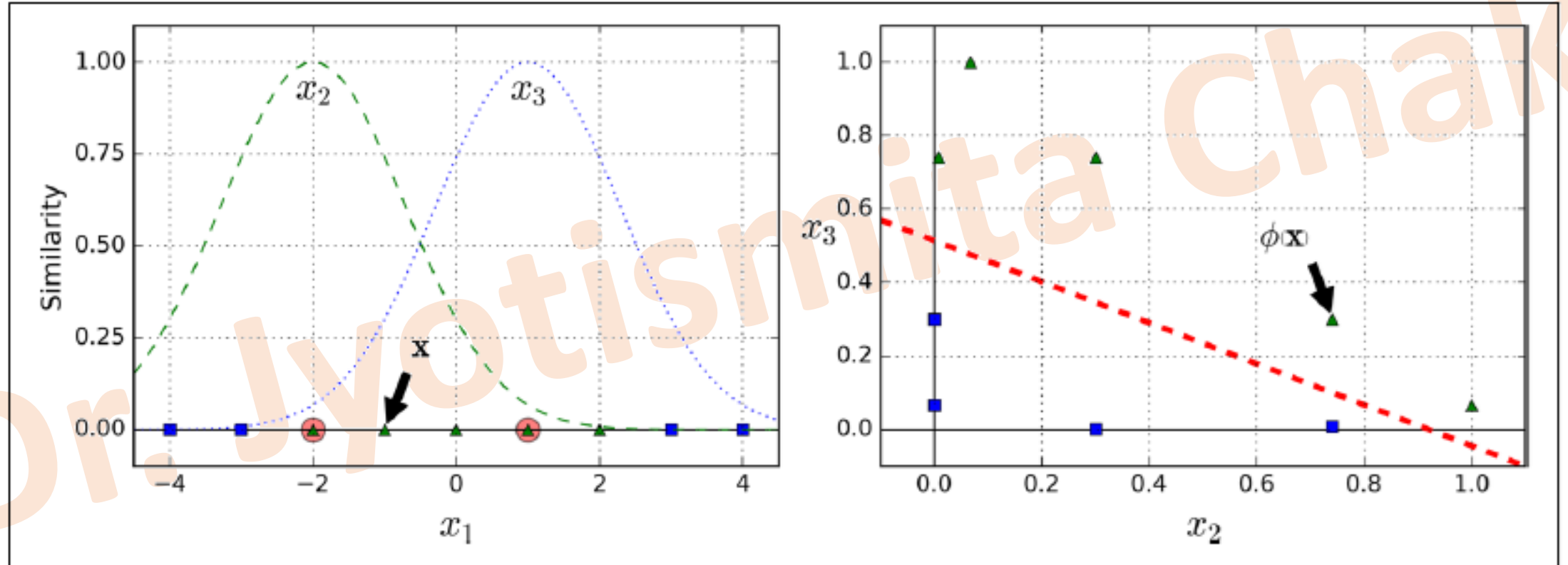


Figure 5-8. Similarity features using the Gaussian RBF

Gaussian RBF Kernel

- The **Radial Basis Function (RBF)** kernel is one of the most powerful, useful, and popular kernels in the Support Vector Machine (SVM) family of classifiers.
- Unlike linear or polynomial kernels, RBF is more complex and efficient at the same time that it can combine multiple polynomial kernels multiple times of different degrees to project the non-linearly separable data into higher dimensional space so that it can be separable using a hyperplane.

Gaussian RBF Kernel

- The RBF kernel works by mapping the data into a high-dimensional space by finding the dot products and squares of all the features in the dataset and then performing the classification using the basic idea of Linear SVM.
- For projecting the data into a higher dimensional space, the RBF kernel uses the so-called radial basis function which can be written as:

$$K(X_1, X_2) = \exp\left(-\frac{\|X_1 - X_2\|^2}{2\sigma^2}\right)$$

Gaussian RBF Kernel

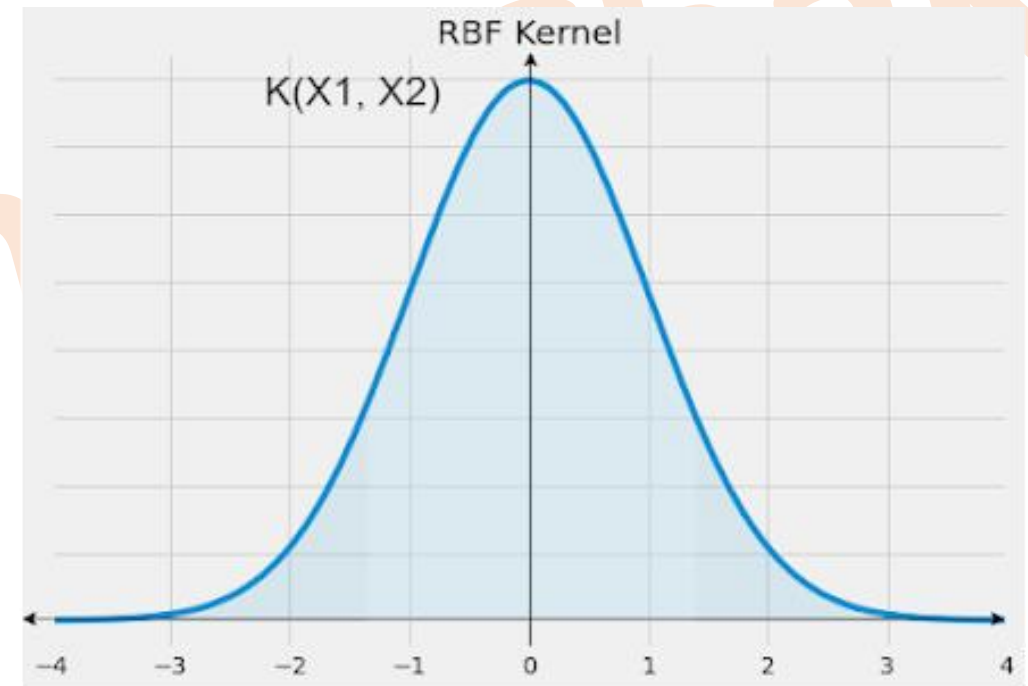
- Here $\|X_1 - X_2\|^2$ is known as the **Squared Euclidean Distance** and σ is a free parameter that can be used to tune the equation.
- When introducing a new parameter $\gamma = 1 / 2\sigma^2$, the equation will be:

$$K(X_1, X_2) = \exp(-\gamma \|X_1 - X_2\|^2)$$

- The equation is really simple here, the **Squared Euclidean Distance** is multiplied by the **gamma** parameter and then finding the **exponent** of the whole.
- This equation can find the **transformed inner products** for mapping the data into higher dimensions directly without actually transforming the entire dataset which leads to inefficiency.
- And this is why it is known as the RBF kernel **function**.

Gaussian RBF Kernel

- The distribution graph of RBF Kernel will look like the figure.
- As you can see that the Distribution graph of the RBF kernel actually looks like the **Gaussian Distribution curve** which is known as a **bell-shaped curve**.
- Thus RBF kernel is also known as **Gaussian Radial Basis Kernel**.



Gaussian RBF Kernel

- Let's try the Gaussian RBF kernel using the SVC class:

```
rbf_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="rbf", gamma=5, C=0.001))
])
rbf_kernel_svm_clf.fit(X, y)
```

- This model is represented on the bottom left of **Figure 5-9**.
- The other plots show models trained with different values of hyperparameters gamma (γ) and C.
- Increasing gamma makes the bell-shape curve narrower, and as a result each instance's range of influence is smaller: the decision boundary ends up being more irregular, wiggling around individual instances.

Gaussian RBF Kernel

- Conversely, a small gamma value makes the bell-shaped curve wider, so instances have a larger range of influence, and the decision boundary ends up smoother.
- So γ acts like a regularization hyperparameter: if your model is overfitting, you should reduce it, and if it is underfitting, you should increase it (similar to the C hyperparameter).
- With so many kernels to choose from, how can you decide which one to use?
 - As a rule of thumb, you should always try the linear kernel first (remember that LinearSVC is much faster than `SVC(kernel="linear")`), especially if the training set is very large or if it has plenty of features.
 - If the training set is not too large, you should try the Gaussian RBF kernel as well; it works well in most cases.
 - Then if you have spare time and computing power, you can also experiment with a few other kernels using cross-validation and grid search, especially if there are kernels specialized for your training set's data structure.

Gaussian RBF Kernel

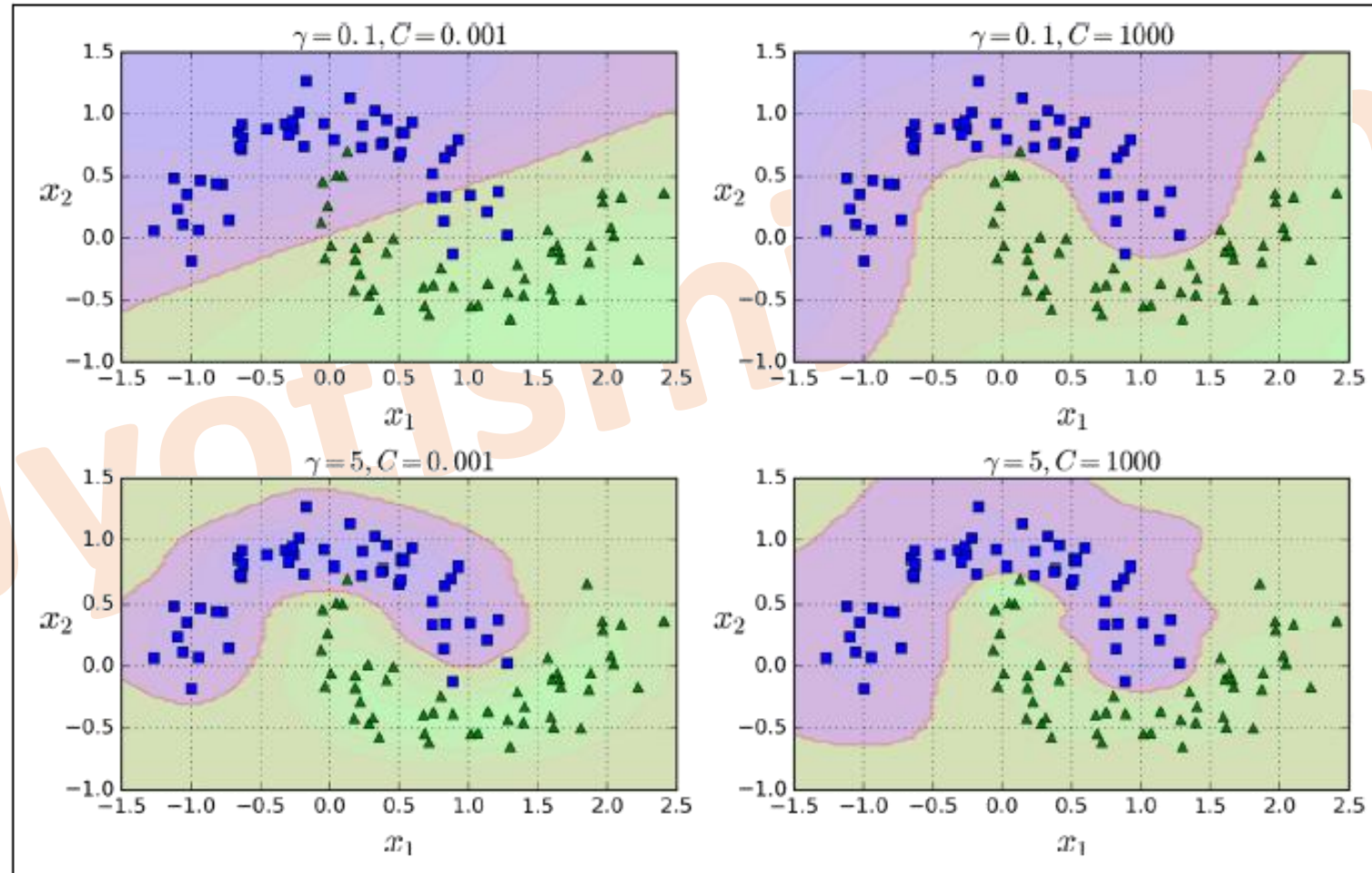


Figure 5-9. SVM classifiers using an RBF kernel

SVM Regression

- The SVM algorithm is quite versatile: not only does it support linear and nonlinear classification, but it also supports linear and nonlinear regression.
- The trick is to reverse the objective: instead of trying to fit the largest possible street between two classes while limiting margin violations, SVM Regression tries to fit as many instances as possible *on* the street while limiting margin violations (i.e., instances *off* the street).
- The width of the street is controlled by a hyperparameter ϵ .
- **Figure 5-10** shows two linear SVM Regression models trained on some random linear data, one with a large margin ($\epsilon = 1.5$) and the other with a small margin ($\epsilon = 0.5$).
- Adding more training instances within the margin does not affect the model's predictions; thus, the model is said to be ϵ -insensitive.

SVM Regression

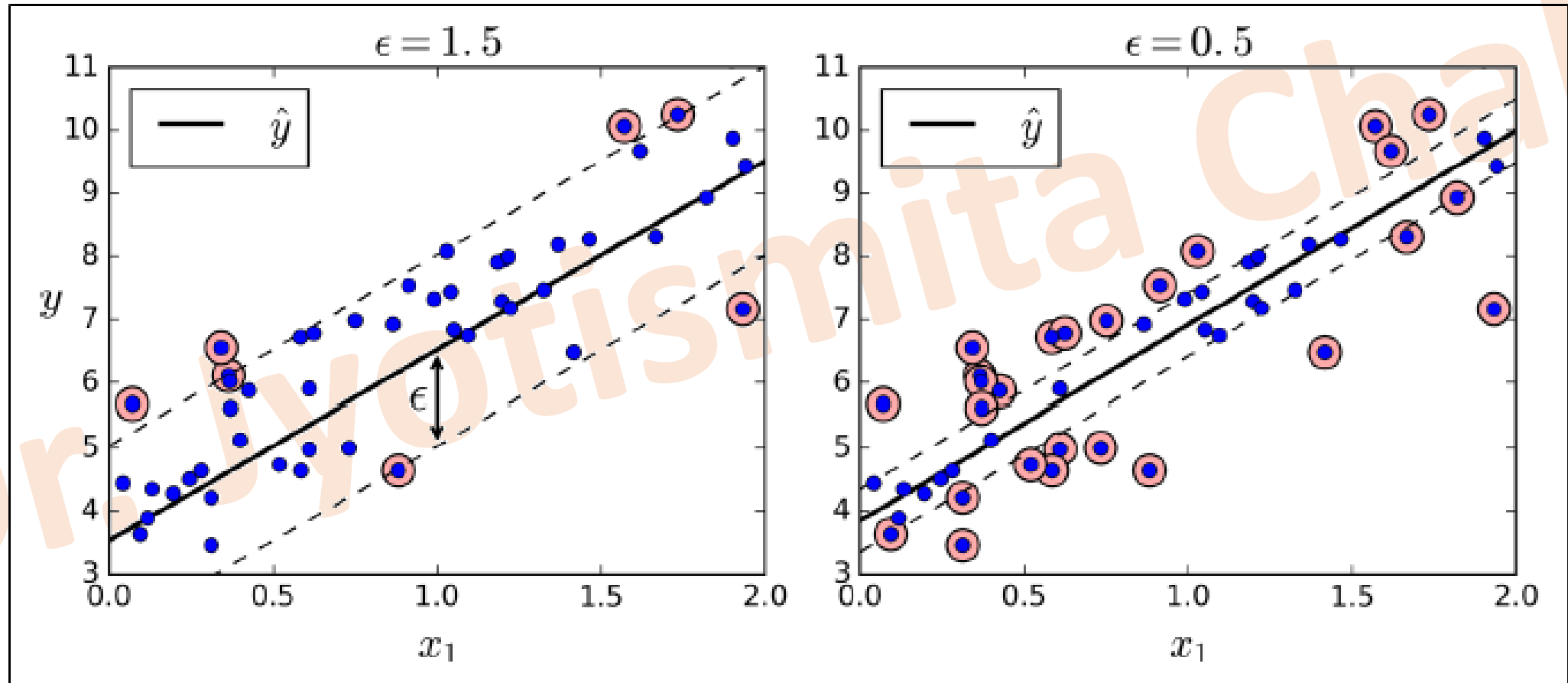


Figure 5-10. SVM Regression

SVM Regression

- You can use Scikit-Learn's LinearSVR class to perform linear SVM Regression.
- The following code produces the model represented on the left of Figure 5-10 (the training data should be scaled and centered first):

```
from sklearn.svm import LinearSVR  
  
svm_reg = LinearSVR(epsilon=1.5)  
svm_reg.fit(X, y)
```

SVM Regression

- To tackle nonlinear regression tasks, you can use a kernelized SVM model.
- For example, **Figure 5-11** shows SVM Regression on a random quadratic training set, using a 2nd-degree polynomial kernel.
- There is little regularization on the left plot (i.e., a large C value), and much more regularization on the right plot (i.e., a small C value).

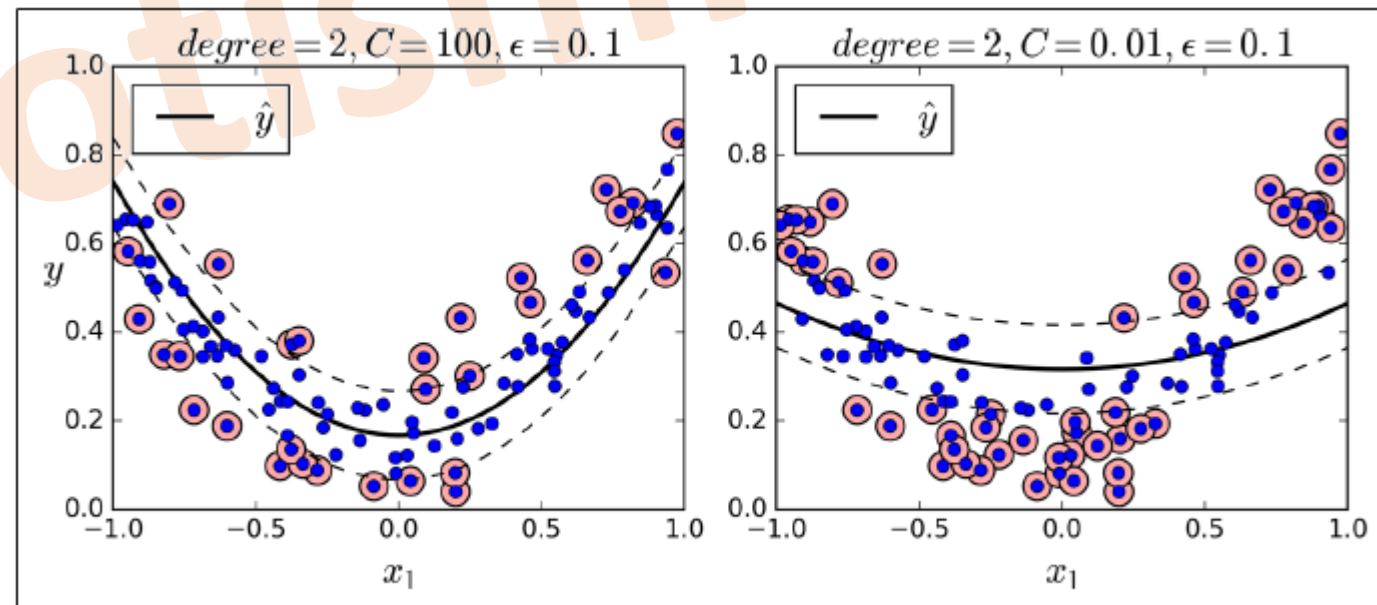


Figure 5-11. SVM regression using a 2nd-degree polynomial kernel

SVM Regression

- The following code produces the model represented on the left of **Figure 5-11** using Scikit-Learn's SVR class (which supports the kernel trick).
- The SVR class is the regression equivalent of the SVC class, and the LinearSVR class is the regression equivalent of the LinearSVC class.
- The LinearSVR class scales linearly with the size of the training set (just like the LinearSVC class), while the SVR class gets much too slow when the training set grows large (just like the SVC class).

```
from sklearn.svm import SVR
```

```
svm_poly_reg = SVR(kernel="poly", degree=2, C=100, epsilon=0.1)  
svm_poly_reg.fit(X, y)
```